NestSelf authentication and authorization: implementation guide

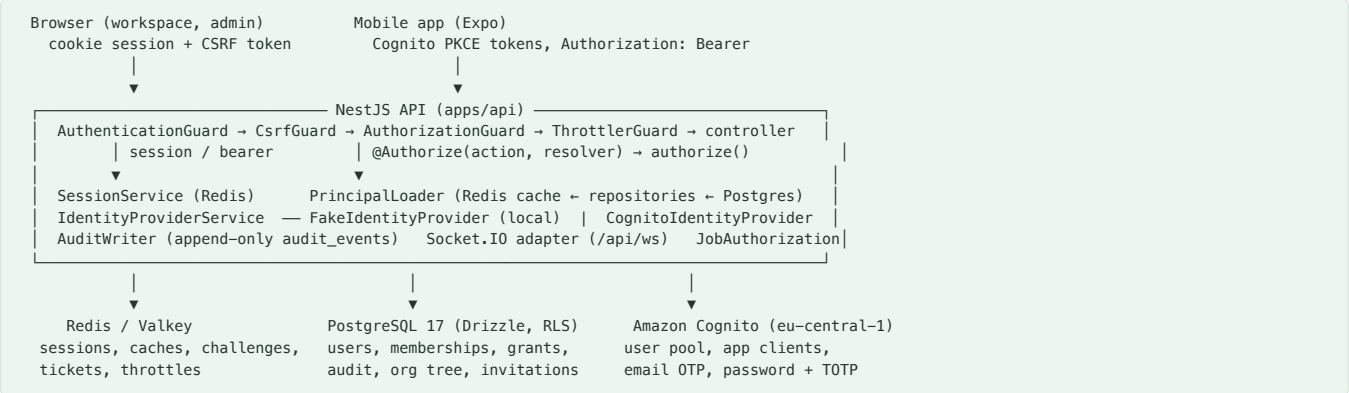
How identity, sessions, authorization, the data layer and the clients fit together, where each piece lives in the repository, and how to work on it. The companion `quick-guide.md` gives the one-page version and the reasoning; `docs/tasks/auth/auth-strategy.md` is the full specification (requirement ids such as `D-06` or `AUTHZ-R-61` below refer to it).

Last updated 2026-10-25 on branch `feat/auth-ui`, after auth phase 2 (plan: `docs/superpowers/plans/2026-10-24-auth-phase-2.md`; status per task: `docs/tasks/auth/phase-2-readiness.md`), the organization owner model (plan: `docs/superpowers/plans/2026-10-24-auth-owner-model.md`) and the UI-completeness plan (`docs/superpowers/plans/2026-10-25-auth-ui-completeness.md`), which gave every auth capability of the API a screen in the web apps. The screenshots are of the real screens, taken by `pnpm docs:auth:screenshots` against the stub API and the seeded sample people (section 19).

Contents

1. The system in one picture
2. Running it locally
3. Repository map
4. The request pipeline
5. Identity provider layer
6. Sign-in flows
7. Sessions, cookies and CSRF
8. Mobile bearer tokens
9. Principal, grants and the authorization engine
10. Data layer: Drizzle, repositories and row-level security
11. Organization and operations administration
12. Audit log
13. WebSockets
14. Background jobs
15. Web clients
16. Mobile client
17. Infrastructure and the Cognito rollout
18. Configuration reference
19. Testing
20. Recipes
21. Known gaps and next steps

1. The system in one picture



Three ideas carry the whole design:

- **Identity is not authorization.** Cognito (or the local fake) only proves who the caller is. What they may do comes from `grants`, `assignments` and `memberships` in PostgreSQL, resolved per request into a `Principal` and evaluated by the pure `authorize()` function. Tokens carry no roles (D-02).
- **Browsers never see tokens.** Web clients hold an opaque session cookie; the API keeps the Cognito tokens encrypted in Redis (Backend-for-Frontend, D-06). Mobile holds Cognito tokens itself and calls the API with a bearer token (D-07).
- **Default deny, twice.** Every route must declare `@Public()`, `@Authenticated()` or `@Authorize()`, enforced by a CI test. Tenant tables also carry PostgreSQL row-level security policies, so a query that forgets its organization filter returns nothing (D-19, D-37).

2. Running it locally

Local development uses `AUTH_MODE=stub`: a fake identity provider in memory, an in-memory key-value store unless `REDIS_URL` is set, and a real PostgreSQL.

```
pnpm install
pnpm db:up                                  # Postgres 17 and Redis 7 via Docker Compose
cp apps/api/.env.example apps/api/.env      # defaults are fine for stub mode
pnpm --filter @nestself/api db:migrate      # apply apps/api/drizzle/*.sql
pnpm --filter @nestself/api db:seed         # fictional fixtures from packages/sample-data
pnpm --filter @nestself/api dev             # http://localhost:4000, Swagger at /api/docs
pnpm --filter @nestself/api worker:dev      # optional: the nightly jobs (needs REDIS_URL)
pnpm --filter @nestself/api job:run grants.reconcile   # run one job once, no queue
```

The seed creates the sample organization, org units, people, grants and an `external_identities` row per person. The fake identity provider is pre-loaded with the same people (`apps/api/src/identity/identity.module.ts`), so sign-in works out of the box:

Sample person	Email	Population	How to sign in
Jovan Petrović	`user_jovan@example.com`	employee	Email code; read it from the API log line `Fake identity provider: OTP for ... is 123456`, or set `FAKE_IDP_FIXED_OTP=123456`
Ivana Jovanović	`user_ivana@example.com`	employer HR admin and employer owner	Email code. The seed gives her `employer_owner` next to `hr_admin` (`EXTRA_SAMPLE_TEMPLATES` in `seed-lib.ts`), which is what opens `/hr/access`
Stefan Marković	`user_stefan@example.com`	team manager	Email code
Nikola Đorđević	`user_nikola@example.com`	operations staff	Password `Nestself-dev-1234!`, then authenticator enrolment on first sign-in
Milica Ristić	`user_milica@example.com`	expert	Password `Nestself-dev-1234!`, then authenticator enrolment
Dragan Podrašćanin	`user_dragan@example.com`	employer HR admin	Email code. Added 2026-10-24 as the developer account linked to the real dev pool; see section 17

For TOTP in development use any authenticator app with the QR code, or compute codes with `computeTotp(secret)` from `apps/api/src/identity/totp.ts`. The web apps run in sample mode by default (`VITE_DATA_SOURCE=sample`); set `VITE_DATA_SOURCE=api` in `apps/workspace/.env.local` or `apps/admin/.env.local` to use the real sign-in screens against the API. Set `SESSION_COOKIE_INSECURE_DEV=true` in the API env so the session cookie works on plain `http://localhost`.

`pnpm db:up` publishes Redis on `6379:6379`, so it fails when a local Redis or Valkey already owns that port. A machine that already runs PostgreSQL and Redis outside Docker needs only `DATABASE_URL` and `REDIS_URL` pointed at them; skip `db:up` entirely.

The worker boots the same `AppModule` as the API, so it needs the API's whole configuration (database, identity provider settings, session encryption key) plus `REDIS_URL`: the `sessions.reconcile-ttl` job revokes refresh tokens through `SessionService`.

`db:seed` grants each sample person only the templates mapped from their legacy role (plus `employee` for every member, plus Ivana's `employer_owner`). Run against a database seeded before 2026-10-24, it also deletes the over-broad sample grants the older seed created, when every person received every template without a legacy role. It stops early when a sample person's `external_identities` row was relinked to a real pool subject, because the upsert then collides on the row id; seed a fresh database instead.

Switching to a real Cognito pool is described in section 17.

3. Repository map

Path	What lives there
`apps/api/src/auth/`	HTTP sign-in layer: `auth.controller.ts`, `authentication.guard.ts`, the sign-in services, challenge store, throttle, bearer verifiers, `step-up.service.ts`, `me-response.builder.ts` (fills `effectiveActions`), `access-revocation.service.ts`, `profile.service.ts`
`apps/api/src/authz/`	Authorization: `authorize.ts` engine and helpers, `authorization.guard.ts`, `authorization.service.ts` (`require()`, `effectiveActions()`), `step-up-freshness.ts`, `throttles.ts`, `authorize.decorator.ts`, `public.decorator.ts`, `authenticated.decorator.ts`, `resolvers.ts`, `context-loader.ts`, golden decision table, `default-deny.test.ts`
`apps/api/src/principal/`	`principal-loader.ts` (user → `Principal` data with Redis caching), `cache-gateway.ts` (circuit breaker), `invalidation-subscriber.ts`
`apps/api/src/session/`	Redis sessions: `session.service.ts`, `session-cookie.ts`, `csrf.guard.ts`, `session-policy.ts` (lifetimes), `token-cipher.ts`
`apps/api/src/identity/`	`identity-provider.service.ts` (abstract contract), `cognito-identity-provider.ts` + helpers, `fake-identity-provider.ts` + `fake-user-store.ts`, `totp.ts`
`apps/api/src/redis/`	`KeyValueStore` contract, Redis and in-memory implementations, `redis-keys.ts` (every key and TTL), `redis-health.indicator.ts`
`apps/api/src/db/`	Drizzle: `db.service.ts`, `schema/*.ts`, `repositories/*.ts`, `seed/` (`seed-lib.ts`, `backfill-grants.ts`), `migrate.ts`
`apps/api/drizzle/`	SQL migrations and drizzle-kit snapshots
`apps/api/src/audit/`	`audit-writer.ts`, `audit-events.ts` (action catalogue and categories), `pino-redaction.ts`
`apps/api/src/invitations/`, `members/`, `org-units/`, `users/`	Organization and operations administration endpoints; `ops-invitations.controller.ts` and `ops-members.controller.ts` serve the `/ops/orgs/:orgId/*` mirrors

Path	What lives there
`apps/api/src/grants/`	Grant administration: `grants.controller.ts` (`/orgs/:orgId/grants`), `approval-authorities.controller.ts` , `ops-grants.controller.ts` (`/ops/grants` , `/ops/orgs/:orgId/grants` , `/ops/users/:userId/access`), `grants.service.ts` , `approval-authorities.service.ts` , `grant-rules.ts` (template, scope, condition and delegation-bound rules), `grant-mapper.ts`
`apps/api/src/professional/`	Confidential-domain reads under `/professional/*` ; `professional-records.ts` holds the resource shape and the response views shared with emergency reads
`apps/api/src/emergency/`	Emergency access request, approval, denial, revocation and record reads under `/ops/emergency-access/*`
`apps/api/src/ws/`	Socket.IO adapter, gateway, handshake authentication, invalidation
`apps/api/src/jobs/` , `apps/api/src/worker/`	`maintenance-jobs.service.ts` (system jobs), `job-scheduler.service.ts` (BullMQ schedules), `job-authorization.ts` (wrapper for principal-bound processors); `worker/main.ts` is the worker entrypoint
`apps/api/src/common/`	`api-error.ts` (error envelope), exception filter, request metadata, Zod pipe
`apps/api/src/config/env.ts`	Zod schema of every environment variable, `AUTH_MODE` rules
`apps/api/test/`	Integration suites against a real PostgreSQL (`*.e2e.spec.ts` , `db/schema.spec.ts`)
`apps/api/docs/auth-core.md`	Route table and module map kept current with the code
`packages/contracts/src/`	Shared Zod schemas and types: `auth.ts` , `principal.ts` , `authorization.ts` , `actions.ts` , `templates.ts` , `org-admin.ts` (grants, approval authorities), `ops-admin.ts` (platform grants, user access, emergency access)
`packages/api-client/src/auth.ts` , `org-admin.ts`	Typed client for every auth endpoint (cookie or bearer mode); `createOrgAdminApi(client, { kind: 'org' | 'ops' })` for the grant administration routes
`apps/workspace/src/auth/` , `apps/admin/src/auth/`	`AuthProvider` , sign-in screens, error mapping, `step-up-dialog.tsx` , `with-step-up.ts` , `route-auth.ts` (route guards wait for the session)
`apps/workspace/src/features/access/` , `apps/admin/src/features/access/`	Access-management screens (section 15)
`apps/{workspace,admin}/src/features/account/` , `features/context/`	Settings (display name, language, sessions, authenticator), the header account menu and the context picker (section 7)
`apps/workspace/src/features/people/` , `features/org-units/` , `features/org/`	HR People (members, invitations, import) and Org units; `features/org/use-org-units.ts` owns the org-units query both these and the access screen read (section 11)
`apps/admin/src/features/organizations/` , `features/people/` , `features/users/` , `features/emergency/`	Operations: organizations list and detail, an organization's people, suspend and reinstate, emergency access (sections 11 and 11.3)
`apps/workspace/src/features/cases/`	Expert area (`/cases`) and the participant's conversations (`/conversations`) (section 11.2)
`apps/api/src/organizations/`	`GET /ops/organizations` and `GET /ops/organizations/:orgId` for the admin organizations screens
`packages/api-client/src/professional.ts`	`createProfessionalApi` : conversation lists and reads, intake reads
`apps/mobile/src/auth/`	Cognito public client (PKCE, email OTP), secure token store, `AuthProvider`
`infra/cdk/lib/auth-stack.ts` , `infra/cdk/lib/triggers/`	Cognito user pool, app clients, Lambda triggers, secrets
`packages/sample-data/src/fixtures/`	Fictional people, org units, credentials, `identities.ts` (fake pool subjects), `grants.ts` (`SAMPLE_GRANTS`), `case.ts` (`SAMPLE_CASE` : one conversation and one intake)

3.1 Screens for each auth capability

Every screen below works against the API (`VITE_DATA_SOURCE=api`) and, in sample mode, over `SAMPLE_*` fixtures with every change disabled and a note saying so. The gates are conveniences; the API enforces every action (section 4). Failures render translated copy (`packages/i18n/locales/*/auth.json` , `workspace.json` , `admin.json`) with an `unknown` fallback, and every step-up action runs through `withStepUp` .

Capability	App and route	Shown when	API	Feature folder
Display name, language	Workspace and admin `/settings`	Signed in	`PATCH /auth/me/display-name` , `PATCH /auth/me/locale`	`features/account/`
Sessions, sign out of one, sign out everywhere	`/settings` , Security	Signed in (not in sample mode)	`GET /auth/sessions` , `DELETE /auth/sessions/:id` , `POST /auth/sign-out-all` , `POST /auth/sign-out`	`features/account/`
Authenticator for a signed-in user	`/settings` , Security	`auth.method === 'password_totp'`	`POST /auth/me/totp/enrol` , `POST /auth/totp/confirm`	`features/account/`
Sign out	Header account menu, both apps	Signed in	`POST /auth/sign-out`	`features/account/user-me`
Context switch	Header context picker, both apps	More than one context available	`POST /auth/context` , then `GET /auth/me`	`features/context/`
Members: list, move, end membership	Workspace `/hr/people` , Members	`program.overview` (move `people.move` , end `people.deactivate`)	`GET` , `PATCH /orgs/:orgId/members/:id` , `POST .../end`	`features/people/`

Capability	App and route	Shown when	API	Feature folder
Invitations: list, invite, import, revoke	Workspace `/hr/people`, Invitations	`people.invite` or `people.import`	`GET`, `POST /orgs/:orgId/invitations`, `POST .../import`, `DELETE .../:id`	`features/people/`
Org units: add, rename, move	Workspace `/hr/org-units`	`org_units.manage`	`GET`, `POST /orgs/:orgId/org-units`, `PATCH .../:id`, `POST .../:id/move`	`features/org-units/`
Grants, delegation, preview, ownership (HR side)	Workspace `/hr/access`	`access.preview` and `program.overview`	`/orgs/:orgId/grants/*`	`features/access/`
Organizations list and detail	Admin `/organizations`, `/organizations/:orgId`	Staff; list needs `client.configure` at platform scope	`GET /ops/organizations`, `GET /ops/organizations/:orgId`	`features/organizations/`
Owner and transfer (staff)	Admin `/organizations/:orgId` Overview	`access.preview`, transfer `access.grant`	`GET /ops/orgs/:orgId/grants`, `POST .../transfer-ownership`	`features/organizations/`
An organization's people	Admin `/organizations/:orgId/people`	`client.configure` (end `people.deactivate`, invitations `people.onboard`)	`/ops/orgs/:orgId/members`, `/ops/orgs/:orgId/invitations`	`features/people/`
Grants (staff side), user access lookup	Admin `/access`, `/organizations/:orgId/access`	`access.preview`	`/ops/grants`, `/ops/orgs/:orgId/grants`, `/ops/users/:userId/access`	`features/access/`
Suspend and reinstate	Admin user lookup result	`people.deactivate`; not offered for one's own account	`POST /ops/users/:userId/suspend`, `.../reinstate`	`features/users/`
Emergency access: request, approve, deny, revoke	Admin `/emergency`	`audit.read` (list), request `access.emergency.request`, decide `access.emergency.approve`	`/ops/emergency-access/*`	`features/emergency/`
Emergency record view	Admin `/emergency`, Open record	The caller's own active access	`GET /ops/emergency-access/:id/records/:type/:id`	`features/emergency/`
Assigned cases (expert)	Workspace `/cases`, `/cases/:conversationId`	`principal.expert` or `conversation.read.assigned`	`GET /professional/conversations`, `GET /professional/conversations/:id`, `GET /professional/intake/:id`	`features/cases/`
My conversations (participant)	Workspace `/conversations`, `/conversations/:id`	`conversation.participate`, not in the expert context	`GET /professional/me/conversations`, `GET /professional/me/conversations/:id`	`features/cases/`

Not covered by a screen: approval authorities (`/orgs/:orgId/approval-authorities`), changing a grant's conditions or validity (`PATCH /orgs/:orgId/grants/:id`), and everything on mobile, which keeps sign-in, settings and sign-out (section 21).

4. The request pipeline

Four global guards run in this order for every HTTP request (`apps/api/src/app.module.ts`):

1. **`AuthenticationGuard`** (`src/auth/authentication.guard.ts`). Skips `@Public()` routes. Otherwise it reads the session cookie or the `Authorization: Bearer` header, resolves a `Principal` and attaches `request.principal`, `request.session` (web only), `request.authMeta` and `request.csrfToken`. Missing or invalid credentials end the request with 401.
2. **`CsrfGuard`** (`src/session/csrf.guard.ts`). Only when a session cookie is present and the method is not safe: it checks `Sec-Fetch-Site`, the `Origin` header against the allowed surfaces and the double-submit token (`X-CSRF-Token` header must equal the CSRF cookie). WebSocket contexts are skipped. Failure is 403 `csrf_rejected`.
3. **`AuthorizationGuard`** (`src/authz/authorization.guard.ts`). Skips `@Public()` and `@Authenticated()` routes. For `@Authorize(action, resolver)` it calls the resolver to derive the `Resource`, builds the context with `ContextLoader`, calls `authorize()`, writes an `access.denied` audit event on denial (the engine's reason is recorded there, never returned to the client) and answers 404 when the caller has no visibility into the organization and 403 otherwise (AUTHZ-R-61). Any principal holding a staff template counts as having visibility, so an operations caller denied in an organization they are not assigned to gets 403 `forbidden`, not 404. When the decision allows and the action has a step-up class, the guard then checks that the authentication is recent enough and answers 403 `step_up_required` otherwise (section 9, Step-up). A handler that carries no decorator at all is denied at runtime as well as by the CI test.
4. **`ThrottlerGuard`** (NestJS throttler, 60 requests per minute by default; sign-in and step-up routes carry tighter `@Throttle` values; grant, approval-authority and emergency-access writes use `WRITE_THROTTLE` from `src/authz/throttles.ts`, 10 per minute per route and client IP).

Decorators (`src/authz/`):

Decorator	Meaning
`@Public()`	No authentication. Sign-in, invitation preview and accept, health, leads.
`@Authenticated()`	A valid session or bearer token, no action check. `/auth/me`, sessions, sign-out, context switch, profile.

Decorator	Meaning
`@Authorize(ACTIONS['x.y'], resolver)`	Full grant-based check. The resolver returns `{ type, id?, orgId?, ... }`; use `orgResourceFromContext(type)` for `/orgs/:orgId/*` routes (it enforces that the active context is that organization, API-R-21), `operationsResource(type, idFn)` for `/ops/*` routes (session required, operations context) and `operationsOrgResource(type)` for `/ops/orgs/:orgId/*` routes (the same checks, plus the route organization in the resource so the engine applies the `assigned_clients` scope through the CLIENT assignment).
`@CurrentPrincipal()`	Parameter decorator returning `request.principal`.

`src/authz/default-deny.test.ts` boots the application and fails when any controller handler or gateway message handler lacks one of the three decorators. It runs with the normal test suite, so an undecorated route cannot reach `main`.

Errors use one envelope (`src/common/api-error.ts`, `api-exception.filter.ts`): `{ status, code, message, requestId, details? }`. Codes the clients map: `unauthenticated`, `session_expired`, `csrf_rejected`, `forbidden`, `not_found`, `context_mismatch`, `session_required`, `invalid_code`, `invalid_credentials`, `invalid_challenge`, `too_many_attempts`, `rate_limited`, `validation`, `conflict`, `duplicate_invitation`, `already_member`, `step_up_required`, `delegation_exceeds_boundary`, `subject_not_member`, `template_inactive`, `missing_conditions`, `self_approval_denied`, `emergency_not_active`, `cannot_suspend_self`, `unavailable` (the list is `AUTH_ERROR_CODES` in `packages/contracts/src/auth.ts`).

5. Identity provider layer

`IdentityProviderService` (`src/identity/identity-provider.service.ts`) is an abstract class used as the DI token. It covers everything the API needs from the identity provider: start and answer the email OTP challenge, password sign-in and the `SOFTWARE_TOKEN_MFA`, `NEW_PASSWORD_REQUIRED` and `MFA_SETUP` challenges, TOTP association and verification, refresh and revocation, global sign-out, enable, disable, create, delete and find users, forgot-password. Sign-in services depend only on this contract, so swapping the provider never touches them.

- **`CognitoIdentityProvider`** (`AUTH_MODE=cognito`): AWS SDK v3 calls with the confidential web client (`SECRET_HASH` on every call). User-management calls live in `cognito-user-ops.ts`; `cognito-error-map.ts` turns Cognito exceptions into `IdentityProviderError` codes.
- **`FakeIdentityProvider`** (`AUTH_MODE=stub`): in-memory users, sessions, OTP codes and refresh-token families with the same challenge chain as Cognito (`fake-user-store.ts`). It refuses to construct when `NODE_ENV=production`, logs email codes, honours `FAKE_IDP_FIXED_OTP`, and is pre-seeded with the sample people in development.

Both return `AuthOutcome`, either `{ kind: 'tokens', tokens }` or `{ kind: 'challenge', challenge, session, username }`.

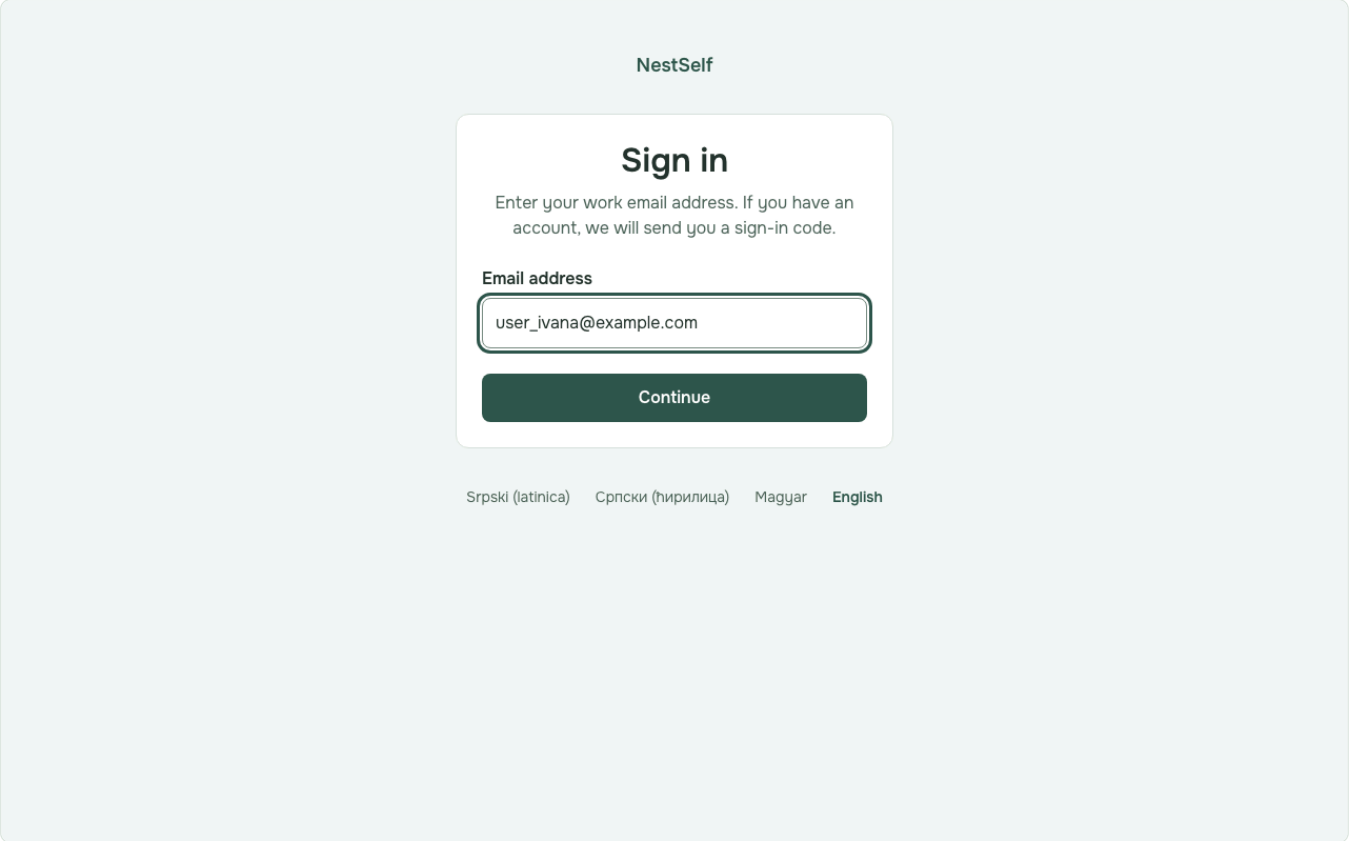
6. Sign-in flows

All flows start at `POST /auth/discover` (`discover.service.ts`), which answers `email_otp`, `password` or `sso` without revealing whether the address exists. Interactive steps are stitched together with **challenge tokens**: `ChallengeStore` (`src/auth/challenge-store.ts`) stores the provider session under `challenge:{sha256(token)}` for 5 minutes and consumes it on every use (`GETDEL`, so two concurrent answers cannot both reach the provider). Failed attempts count in `SignInThrottle` (`throttle:otp:{email}`): five failures lock the identifier for 15 minutes with 429 `too_many_attempts`. Unknown addresses go through the same timing (`neutral-delay.ts`) and receive a dummy challenge so responses do not reveal existence.

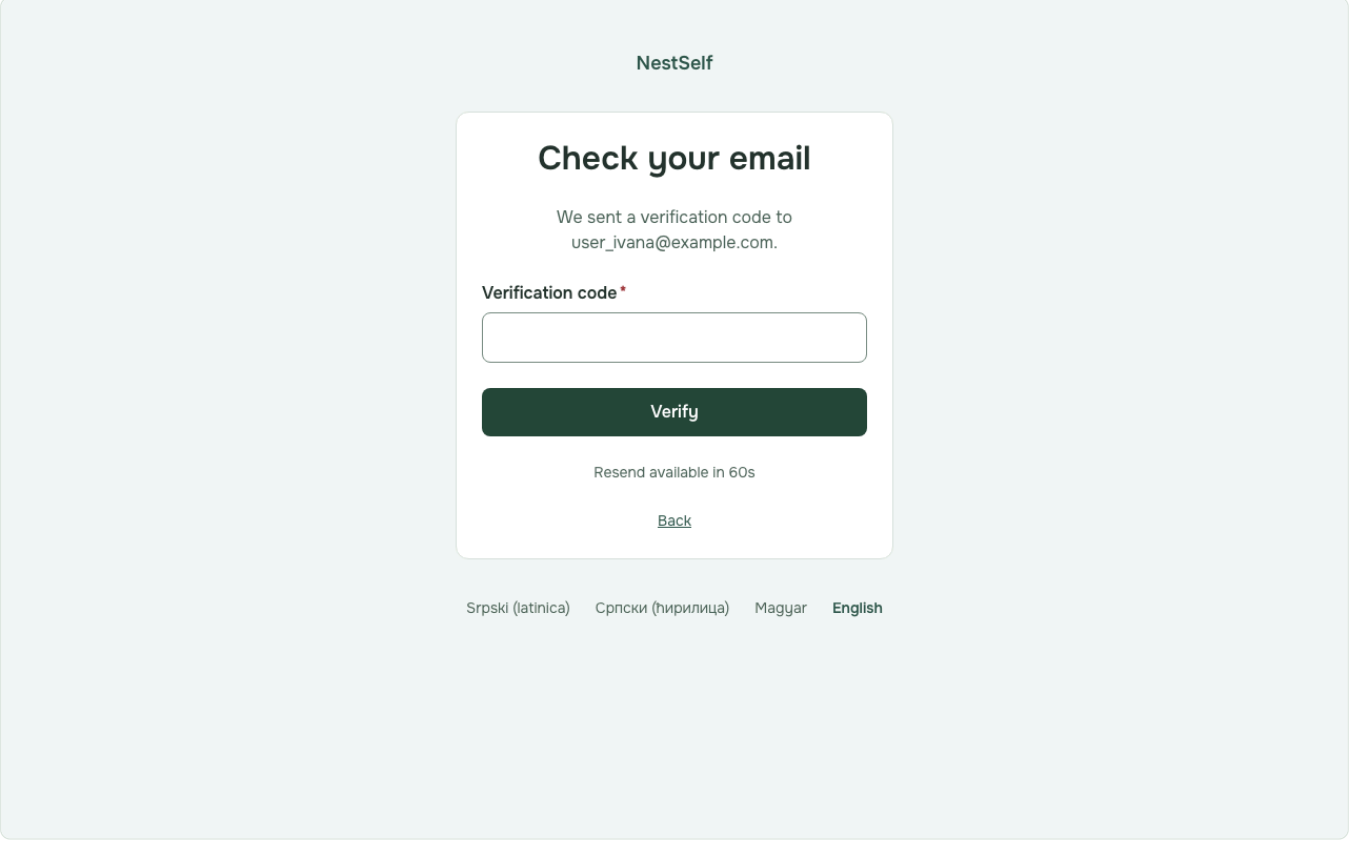
Cognito's EMAIL_OTP code is eight digits, unlike the six-digit codes it issues for sign-up confirmation, forgot-password and authenticator apps, and unlike the stub provider's `FAKE_IDP_FIXED_OTP`. `EMAIL_OTP_MIN_LENGTH` and `EMAIL_OTP_MAX_LENGTH` in `packages/contracts/src/auth.ts` hold that range; `EmailOtpVerifyRequestSchema`, `OtpInput` and the mobile field all read it. Before 2026-10-24 every one of them hard-coded six, so the input truncated the real code, the user could not type the last two digits, and Cognito answered `CodeMismatchException`.

A wrong email code can be retyped, up to three times per challenge. `emailOtpVerify` and the step-up `verify` consume the challenge before asking the provider. When the provider answers `invalid_code` (Cognito's `CodeMismatchException`, which keeps its own `Session` alive), `retainAfterWrongCode` in `sign-in.service.ts` puts the record back under the same token with `attempts + 1` through `ChallengeStore.put()`, and the API answers 401 `invalid_code`; the user corrects the code without requesting a new one. `put` keeps the deadline stamped at creation (`expiresAt`), so retries never extend the five minutes. The third wrong code (`MAX_EMAIL_OTP_ATTEMPTS = 3`) is not put back: the next attempt with that token answers 401 `invalid_challenge` and the screens ask for a new code. Any other provider error ends the challenge at once. Dummy challenges for unknown addresses follow the same budget so the answers stay identical. `SignInThrottle` still counts every wrong code, across challenges, and locks the address after five. Each wrong code is counted before the record is handed back, and the record that triggers the lock is not handed back; a challenge retained earlier answers 429 `too_many_attempts` once the address is locked (checked after `consume` for dummy and real challenges alike), so a lock cannot be outrun with a token that still has tries left.

Employees: email code. `POST /auth/email-otp/start` with `{ email, locale? }` → `{ challengeToken }` (`locale` is one of `en`, `sr-Latn`, `sr-Cyrl`, `hu`; the API forwards it to the provider as `ClientMetadata.locale`, which the CustomMessage trigger uses for the email's language when the user has no `custom:locale` attribute; the web screens send the language they are shown in and the mobile app sends its resolved device locale directly to Cognito); `POST /auth/email-otp/verify` with `{ challengeToken, code }` → tokens → `IdentityResolutionService.resolveIdentityAndCreateSession()` maps the token `sub` to an `external_identities` row, loads the principal, creates the Redis session, sets the cookies and returns `{ principal, effectiveActions }`.

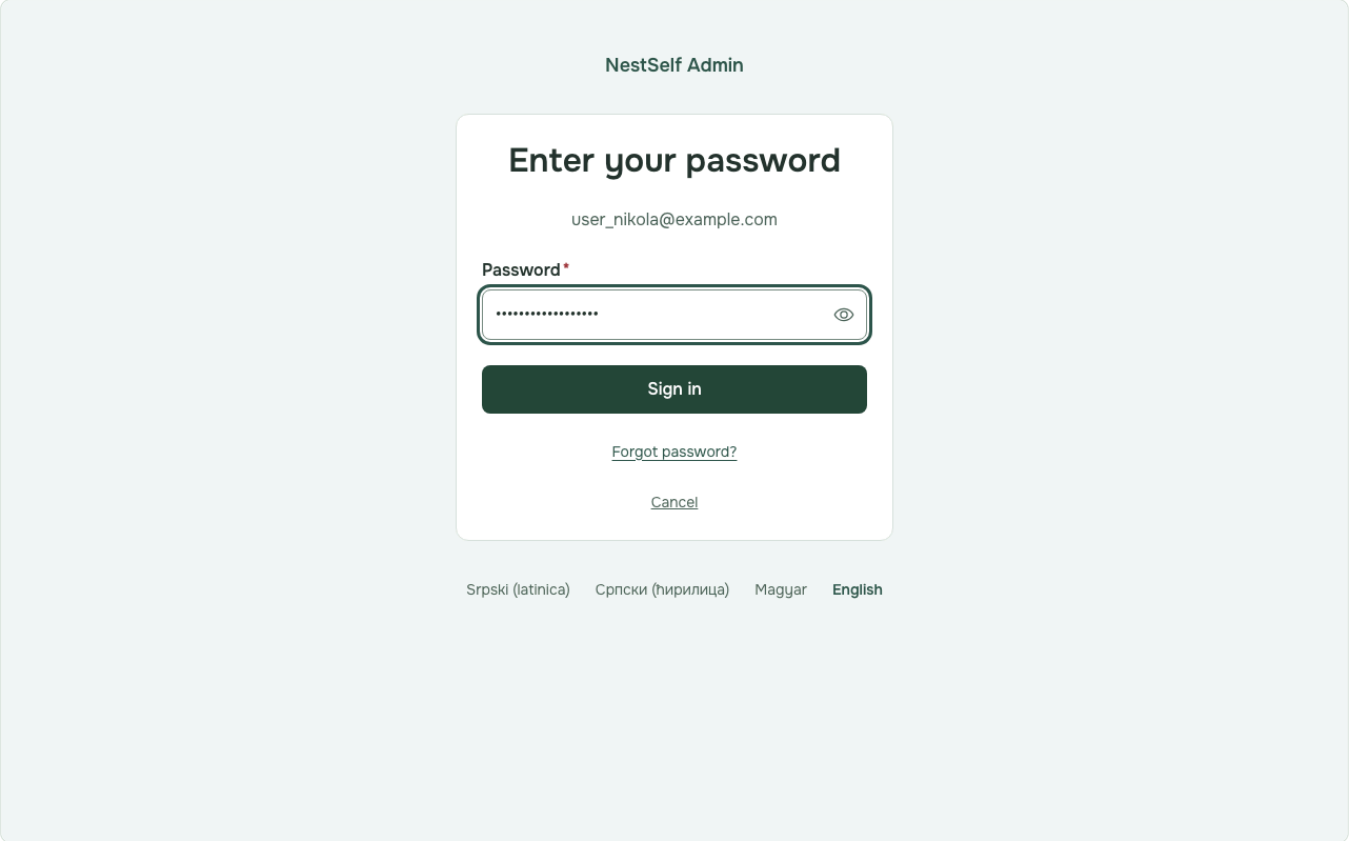


Workspace sign-in, first step: the address decides the method (`POST /auth/discover`).

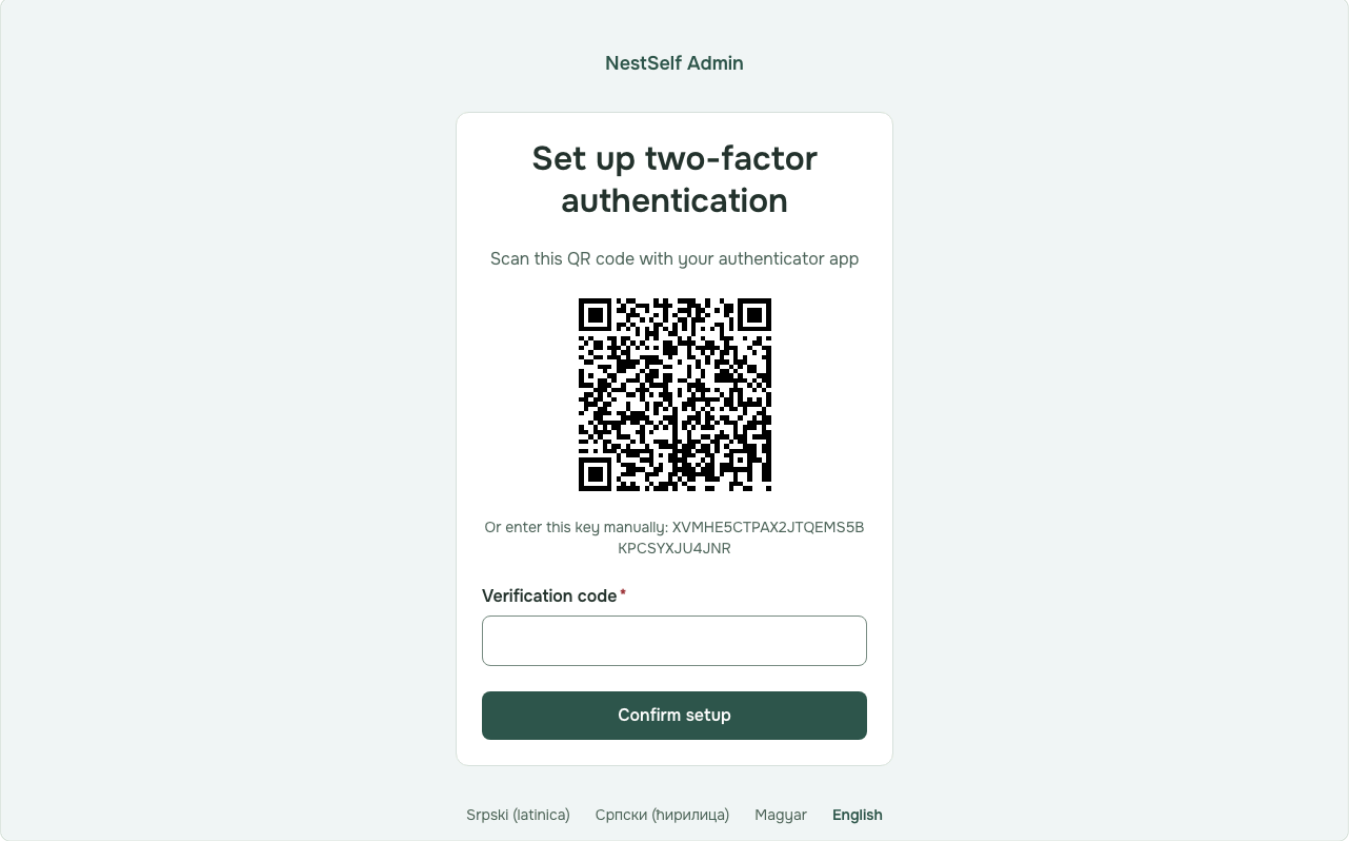


The email-code step. The field takes the code length range from `@nestself/contracts` (eight digits from Cognito).

Experts and staff: password and authenticator. `POST /auth/password` returns either the signed-in principal or `{ step, challengeToken }` where `step` is `new_password` , `totp` or `totp_enrol` . `POST /auth/challenge` with `{ challengeToken, response }` answers one step and returns either the next `{ step, challengeToken }` (200) or the principal. For `totp_enrol` the client first calls `POST /auth/totp/enrol` with the challenge token to receive the `otpauth://` URI and a rotated challenge token, then `POST /auth/challenge` with the authenticator code; the API verifies the software token and completes `MFA_SETUP` in one step (`sign-in.service.ts` , `totp.service.ts`). `POST /auth/totp/confirm` is the signed-in variant for adding an authenticator later.



Admin sign-in for staff and experts: the password step after the identifier.



First sign-in: enrolment shows the `otpauth://` URI as a QR code and as a manual key, then takes the first authenticator code (`totp_enrol`). Later sign-ins show only the code field.

Password reset. `POST /auth/password-reset/start` and `/confirm` (`password-reset.service.ts`) wrap the provider's forgot-password flow with neutral responses.

Invitation acceptance. `GET /invitations/:token/preview` returns only the organization name; `POST /invitations/accept` (`src/invitations/invitation-activation.service.ts`) validates the hashed token, creates or links the user, ensures the identity-provider user and the `external_identities` row, creates the

ACTIVE membership and the template grants in one transaction, then starts the right sign-in method: an email code for employees or a temporary-password flow for staff and experts.

SSO. `GET /auth/sso/start` and `/callback` are not implemented yet (phase 3); the discover step still reports `sso` when a verified domain matches an active connection.

Every sign-in writes `auth.sign_in.succeeded` or `auth.sign_in.failed` through `AuditWriter`.

7. Sessions, cookies and CSRF

`SessionService` (`src/session/session.service.ts`) stores one JSON record per session at `session:{sha256(sessionId)}` and indexes a user's sessions in `user_sessions:{userId}`. The record (`session.types.ts`) holds the user id, surface (`workspace` or `admin`), population, active context, authentication metadata, lifetimes, a hashed IP and user-agent family, the CSRF token, and the Cognito tokens encrypted with AES-256-GCM (`token-cipher.ts`, key `SESSION_ENCRYPTION_KEY`, dual-key rotation via `SESSION_ENCRYPTION_KEY_PREVIOUS`).

Cookies (`session-cookie.ts`): `__Host-nestself_session` and `__Host-nestself_csrf`, `Secure`, `HttpOnly` for the session, `SameSite=Lax`, `Path=/`. `SESSION_COOKIE_INSECURE_DEV=true` drops the `__Host-` prefix and `Secure` for plain-HTTP localhost. The CSRF cookie is readable by JavaScript on purpose: the API client copies it into `X-CSRF-Token`.

Lifetimes (`session-policy.ts`) depend on population and context, for example employee personal 30 minutes idle and 12 hours absolute, employee organization 60 minutes and 8 hours, expert 8 hours and 7 days, staff 60 minutes and 8 hours; `SESSION_LIFETIME_OVERRIDES` (JSON) adjusts them per environment within the 30-day idle and 90-day absolute caps. Each request extends the idle window; the absolute expiry never moves. The session id is regenerated at sign-in and again after a completed step-up (section 9).

No ETags on API responses. `main.ts` calls `set('etag', false)` on the Express instance. Express otherwise emits a weak ETag for every JSON response, the browser revalidates with `If-None-Match`, and the API answers `304 Not Modified` with no body. `@nestself/api-client` treats anything outside 2xx as a failure (`if (!response.ok)`), so a page refresh turned a live session into an `ApiError` with status 304 and bounced the user to `/sign-in` while the Redis session was still healthy. These responses are per-principal and already `Cache-Control: no-store`; they must never be revalidated.

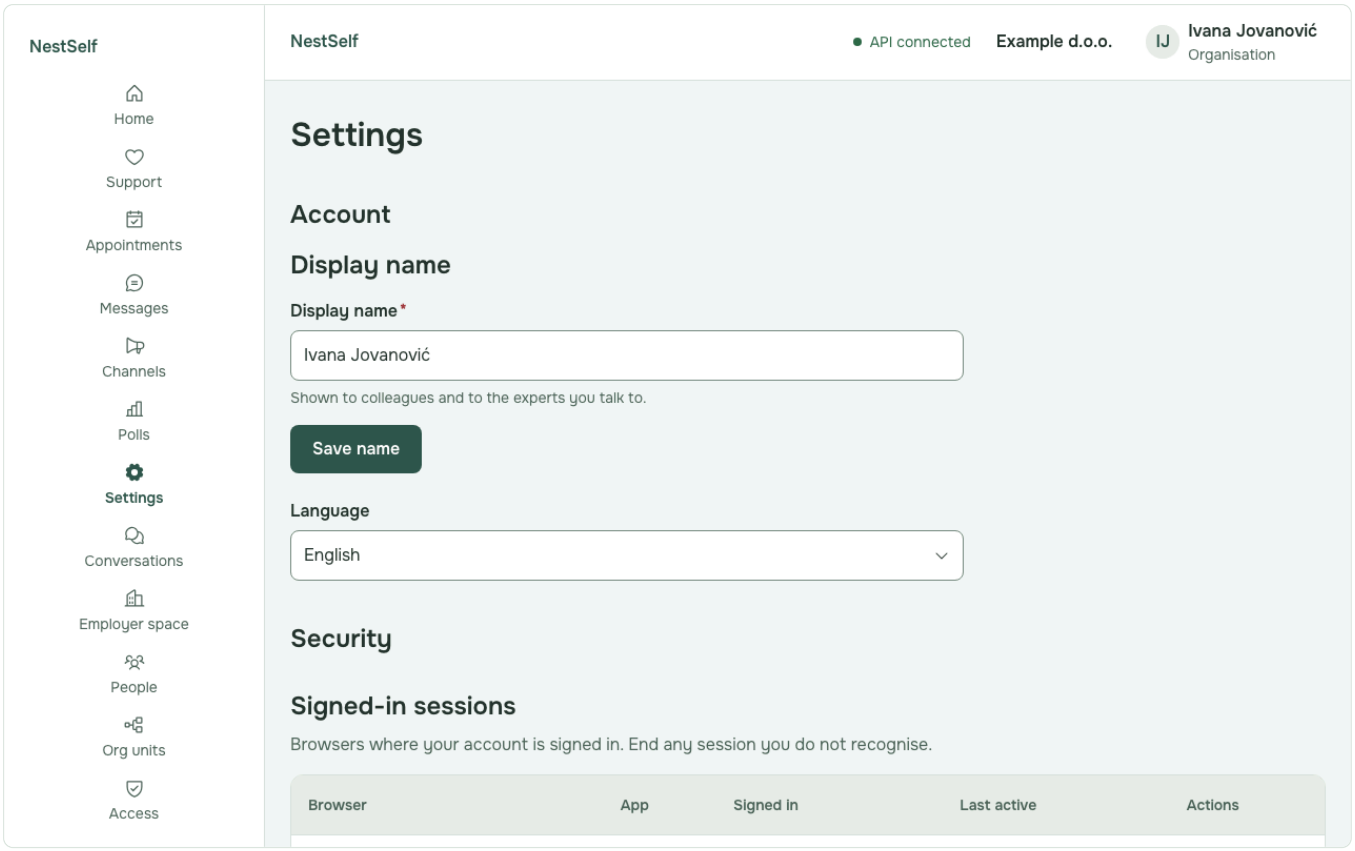
Access-token refresh happens lazily: `ensureFreshAccessToken` refreshes through the identity provider under `lock:session:{hash}` so concurrent requests do not race; a failed refresh destroys the session.

Revocation paths: `POST /auth/sign-out` (current session), `POST /auth/sign-out-all` (every session, Cognito global sign-out), `DELETE /auth/sessions/:id` (another device), and `AccessRevocationService.revokeAllAccess()` (`src/auth/access-revocation.service.ts`), which membership end and suspension call: destroy sessions, `globalSignOut` for every external identity, publish the principal invalidation. It never throws; it reports what failed.

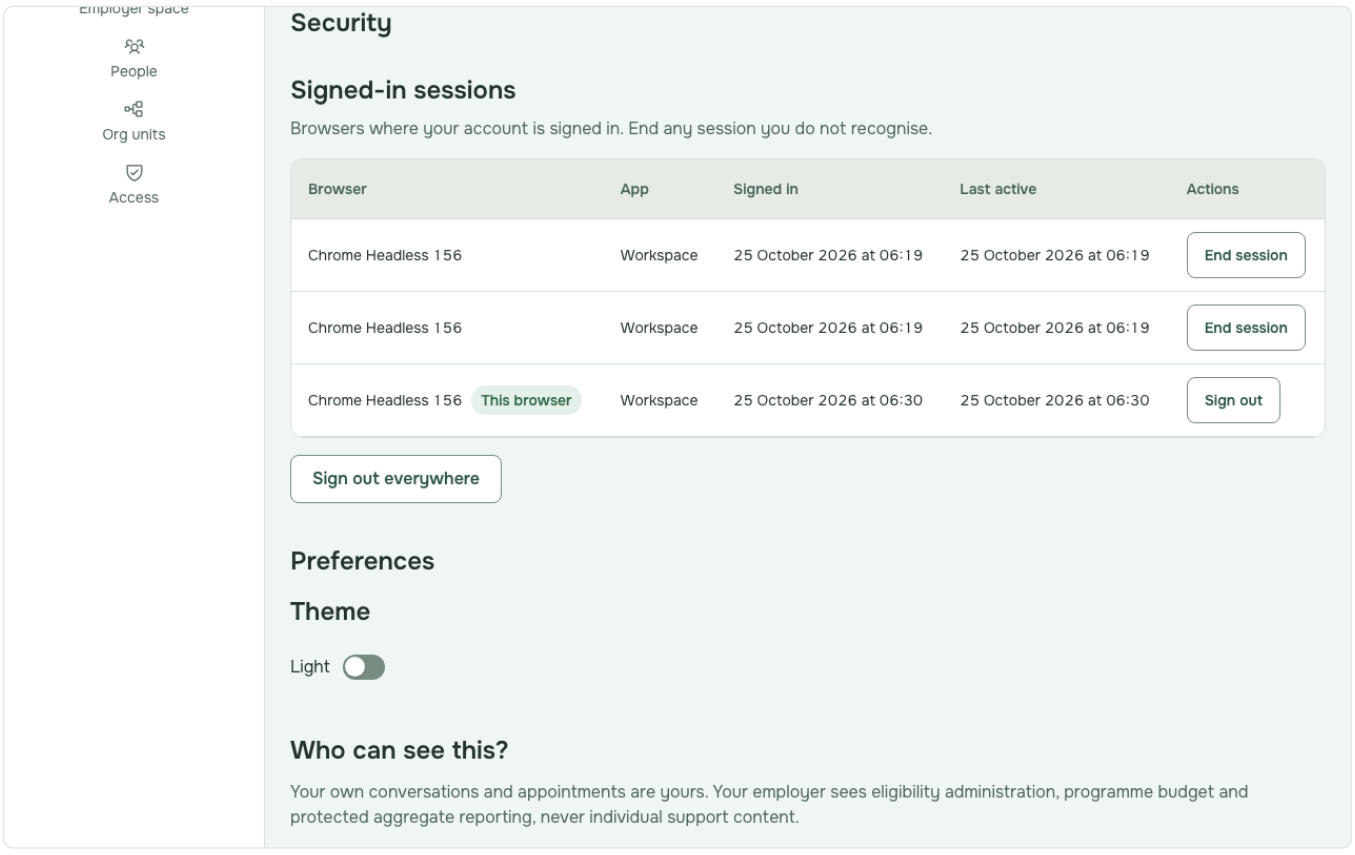
Context switch (`POST /auth/context`, `session-endpoints.service.ts`) changes the active context (`personal`, `organization`, `expert`, `operations`) after checking the principal actually holds it; it never extends the absolute expiry.

Account security screens. `/settings` in both web apps (`features/account/`) has three parts. *Account*: the display name (`PATCH /auth/me/display-name`, at most `DISPLAY_NAME_MAX_LENGTH` = 120 characters from `@nestself/contracts`) and the language, saved to the account and then applied to the screen. *Security*: the signed-in sessions (`GET /auth/sessions`), with the current browser marked; ending another session calls `DELETE /auth/sessions/:id`, while the row of the current browser reads "Sign out" and calls `POST /auth/sign-out`, because the API refuses to delete the current session with 409. "Sign out everywhere" calls `POST /auth/sign-out-all`, which keeps the caller's own session, and then `POST /auth/sign-out`, so this browser ends too and lands on `/sign-in`; if the first call fails the user stays on the page with the message. Staff and experts who sign in with a password also see the authenticator section: `POST /auth/me/totp/enrol` (refused with 409 `totp_already_enrolled` when an authenticator exists) and `POST /auth/totp/confirm`; the secret never outlives the screen (`gcTime: 0` on both mutations and a reset on cancel and unmount). *Preferences*: the theme. The header of both apps carries the account menu (name, current context, Settings, Sign out).

The display-name field shows the saved name until the user types, also on a deep link or a reload of `/settings`, where the principal arrives after the first render (fixed 2026-10-25; the field used to stay empty).



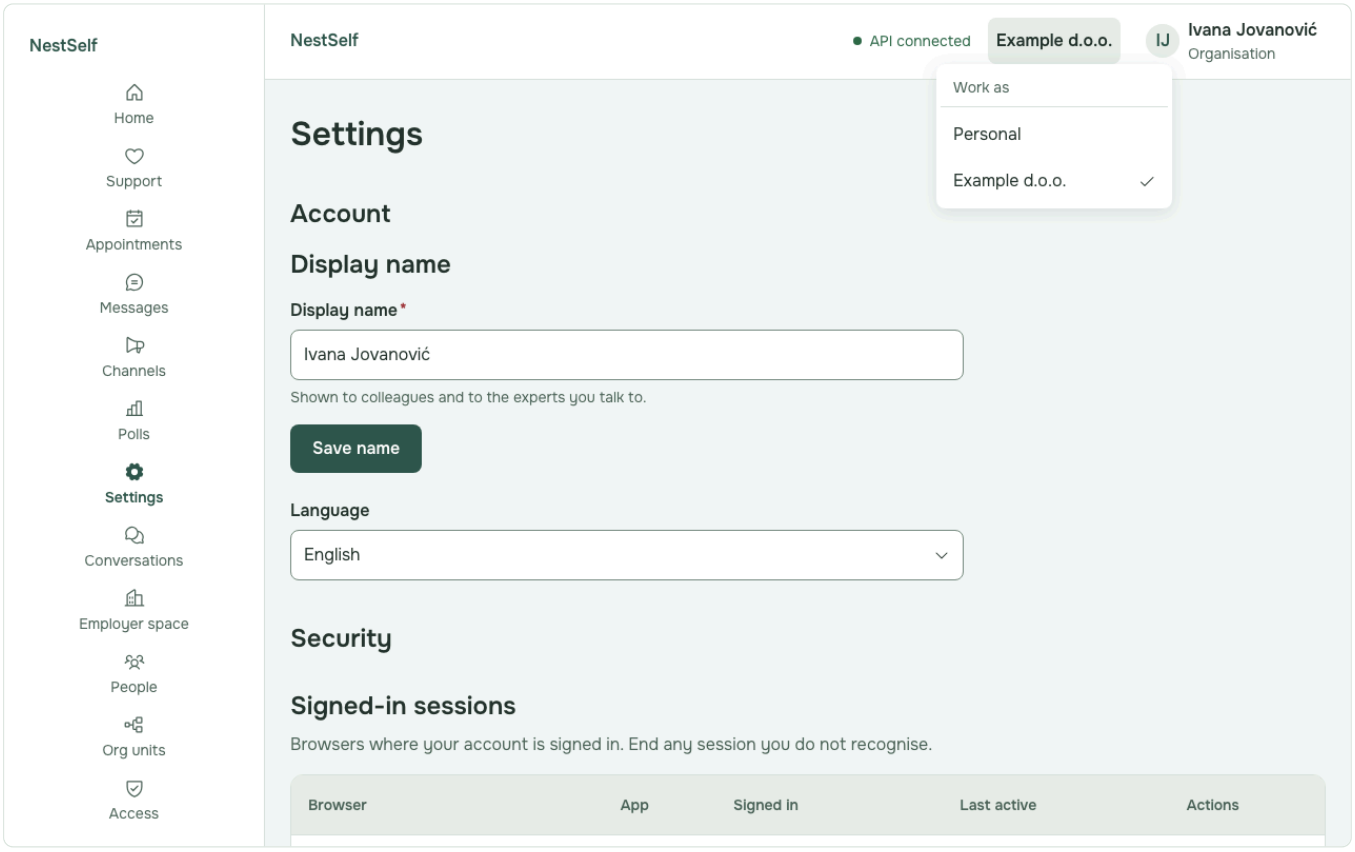
Workspace `/settings` for Ivana Jovanović: display name and language are saved to her account.



The sessions list: one row per signed-in browser, "This browser" on the current one, End session on the others and Sign out everywhere below.

Context picker. `features/context/context-picker.tsx` sits in the header of both apps, before the account menu, whenever the principal has more than one context: Personal, each organization with an ACTIVE membership (by `orgName`, which `/auth/me` memberships carry since 2026-10-25), Expert when `principal.expert` is set and Operations when the principal holds a staff template. Choosing one calls `POST /auth/context` and then `GET /auth/me`, so `effectiveActions` and the gated navigation change without a reload; the workspace then opens `/` (or `/cases` for Expert), the admin app `/`. On `invalid_context` or `org_context_expired` the picker refreshes the principal, so an ended membership leaves the menu. Failures show a toast with translated

copy. The on-mount context hooks of the people, access, operations and expert screens switch at most once per mount, so they never undo a choice made in the picker.



The context picker open for Ivana: Personal or her organization, Example d.o.o. (current).

8. Mobile bearer tokens

Mobile apps authenticate against Cognito directly and send `Authorization: Bearer <access token>`. `AuthenticationGuard` passes the token to the `BEARER_VERIFIER` provider (`src/auth/bearer-verifier.ts`): `CognitoBearerVerifier` verifies signature and claims with `aws-jwt-verify` (`tokenUse: 'access'`, the mobile client id) and rejects tokens whose refresh family is listed in `revoked_family:{originJti}`; `StubBearerVerifier` accepts HS256 tokens signed with `JWT_SECRET` in stub mode. The guard then maps `sub` to the user through `external_identities`, records the family in `user_mobile_families:{userId}` and refreshes `mobile_family:{originJti}` (30-day TTL, AUTHZ-R-64) and assembles the principal with the default context for the user. Bearer callers cannot use `/ops/*` routes (API-R-28) and get 403 `session_required`. The nightly `mobile.prune-stale-families` job (section 14) denylists families whose last-seen key has expired.

Cognito access tokens carry `auth_time` but not the method, so `CognitoBearerVerifier` reports `email_otp` with no second factor. That is accurate today: the mobile app client only permits the email-code first factor (`USER_AUTH` with `EMAIL_OTP`), passwords and authenticators are web-only, and SSO (which would set `sso`) is phase 3. Bearer callers step up through the same `/auth/step-up/*` endpoints as the browser, never through claims; the result is stored under `stepup:{tokenFamily}` for 30 minutes (`STEP_UP_BEARER_TTL`). The mobile app has no step-up screen yet (section 21).

9. Principal, grants and the authorization engine

Shape. `Principal` (`packages/contracts/src/auth.ts`, `principal.ts`): `userId`, `displayName`, `initials`, `locale`, `memberships[]` (id, orgId, orgUnitId, status, dates), `staffTemplates[]`, `expert` (id and approved categories), `context`, `auth` (method, time, second factor, last step-up), `sessionId`, `tokenFamily`. It carries no grants; those are loaded per decision.

Loading and caching. `PrincipalLoader` (`src/principal/principal-loader.ts`) reads through the repositories and caches in Redis with jittered TTLs: `principal:{userId}` 30 s, `grants:{userId}:{orgId}` 60 s, `assignments:{userId}` 60 s, `org_tree:{orgId}` 5 min. `CacheGateway` wraps Redis with a circuit breaker (three failures open it for 5 s) and a small in-memory LRU fallback. Writes that change access call `invalidateUser(userId, orgIds)`, which deletes the keys and publishes on the `invalidate` channel; `InvalidationSubscriber` on every API instance drops its local copies, and the WebSocket layer re-authorizes connected sockets.

Delegated grants carry their ancestors. When the subject holds a delegated grant, `loadGrants` also loads its ancestor chain (`AuthorizationRepository.findDelegationAncestors`, a recursive walk over `parent_grant_id` in system context, revoked rows included) into the same `grants:{userId}:{orgId}` entry, because `validateDelegation` needs the whole chain to decide. A change to an ancestor reaches that entry only through the delegatee's own invalidation, so every path that revokes or adjusts a grant calls `PrincipalLoader.invalidateDelegatees(grantIds)`, which walks the tree downwards (`findDelegationDescendants`) and invalidates every subject below: grant revoke and update, membership end and the `grants.reconcile` job. It runs after the commit; if it fails, delegatees keep stale copies until the grants TTL ends.

Vocabulary. `ACTIONS` (`packages/contracts/src/actions.ts`) is the closed catalogue of dotted action strings (`people.invite`, `program.overview`, `conversation.read.assigned` ...). `PERMISSION_TEMPLATES` (`templates.ts`) are the 17 named starting sets (`platform_owner`, `client_operations`, `hr_admin`, `team_manager`, `employee`, `expert` ...) with default actions, scope type, conditions and step-up actions. A **grant** gives a user a list of actions within a scope (`own`,

`org_unit` with optional descendants, `organization`, `platform`, `assigned_cases` ...) with optional Zod-validated conditions (spending ceiling, categories, confidential approver, time window), a validity window and, for delegated grants, a parent and depth limit of 3.

Deciding. `authorize(principal, action, resource, ctx)` (`src/authz/authorize.ts`) is pure and ordered: user state (suspended or deactivated denies everything), context validity (`context.ts`), confidential hard deny, assignments, grants with scope matching through the closure-table ancestor set (`scope.ts`), conditions (`conditions.ts`), validity window, self-approval rule, delegation envelope (`delegation.ts`). It returns `{ allowed, grantIds, assignmentIds, denialReason, explanation[] }`; `explain.ts`, `authorize-many.ts` and `effective-access.ts` build on it. `ContextLoader` (`context-loader.ts`) assembles the `AuthorizationContext` (grants, assignments, ancestor ids, organization settings, emergency access for confidential actions, which is never cached). Step 5 matches only grants whose `subjectUserId` is the principal's own; the ancestors loaded for a delegated grant serve `validateDelegation` and never lend their actions to the delegatee. The golden table `golden-table.json` (150 tuples, including the `CONF-DENY-01` to `06` confidential hard-deny rows and the `DELEG-ALLOW-01`, `DELEG-DENY-01` to `03` delegation rows) and fast-check property tests (`authorize.property.test.ts`) pin the behaviour: no cross-organization allow, delegation never exceeds the delegator, expired never allows, confidential never allowed by administrative scopes.

Using it outside a guard. `AuthorizationService.require(principal, action, resource)` is for service-layer checks whose resource needs row data a static resolver cannot load (the confidential domain and emergency record reads, section 11). It audits denials like the guard, maps them to 403 or 404 the same way, and applies the same step-up check. `effectiveActions(principal)` fills `effectiveActions` in every `MeResponse` through `MeResponseBuilder` (`src/auth/me-response.builder.ts`): sign-in completion, `GET /auth/me`, `POST /auth/context` (for the new context) and the two profile updates. Before 2026-10-24 every response carried an empty list, so every action-gated screen was unreachable in api mode. An action counts when any current grant allows it, and an `org_unit` grant is evaluated at its own unit, so team-scoped templates such as `team_manager` appear. Clients get the same answer client-side as a hint through `can()` in `packages/contracts/src/principal.ts`; it is never the enforcement.

Step-up. `STEP_UP_CLASS_BY_ACTION` (`packages/contracts/src/authorization.ts`) puts sensitive actions into five classes, each with a maximum age in `STEP_UP_MAX_AGE_SECONDS`:

Class	Maximum age	Actions
`emergency`	5 min	`access.emergency.request`, `access.emergency.approve`
`restricted`	15 min	`conversation.read.assigned`, `notes.read.supervision`, `records.read.emergency`
`export`	15 min	`report.export`, `data.export.own`
`financial`	15 min	`spend.approve`, `funding.allocate`, `budget.allocate`
`administration`	30 min	`access.grant`, `access.revoke`, `access.delegate`, `people.deactivate`, `sso.manage`, `session.revoke.any`

`assertStepUpFresh` (`src/authz/step-up-freshness.ts`) runs after an allow decision, both in `AuthorizationGuard` and in `AuthorizationService.require()`. It takes the latest of `auth.time`, `auth.lastStepUpAt` and, for bearer callers, the `stepup:{tokenFamily}` record; when that is older than the class allows it throws 403 `step_up_required` with `details: { stepUpClass, maxAge, method: 'email_otp' }`. A caller who signed in again after an old step-up counts from the new sign-in. Denied callers keep their 403 or 404 and are never asked to step up. The `restricted` class is enforced as a plain 15-minute maximum age, not as "first access per session" as the strategy words it; the stricter reading can be added without a contract change.

`StepUpService` (`src/auth/step-up.service.ts`) serves `POST /auth/step-up/start` (200, `{ method: 'email_otp', challengeToken, ssoStartUrl: null }`) and `POST /auth/step-up/verify` (`{ challengeToken, code }`, 204). Both are `@Authenticated()` and limited to 5 requests per minute. Start sends an email code to the address of the principal's first Cognito identity and stores a challenge that records `stepUp.userId`; a principal without a native identity (federated users, until phase 3) gets 403 `forbidden`. Verify refuses a challenge minted for another user, or for sign-in, with 401 `invalid_challenge` (another user's step-up challenge is also audited as `auth.step_up.failed` with `reason: 'foreign_challenge'`, never the token or the code), and `/auth/email-otp/verify` refuses step-up challenges the same way, so a step-up code can never mint a new session. A wrong code follows the three-attempt rule of section 6. On success a session caller gets `markStepUp`, then `regenerate`: the 204 carries a new session cookie and CSRF token. A bearer caller gets `stepup:{tokenFamily}` with a 30-minute TTL. The throwaway refresh token the provider issued is revoked, and `auth.step_up.completed` or `auth.step_up.failed` is audited.

Every class steps up with an email code, staff and experts who signed in with a password and an authenticator included, and the `emergency` class included, for which the strategy asks for an authenticator. Cognito's `USER_AUTH` flow cannot issue a `SOFTWARE_TOKEN_MFA` challenge without a password first, so an authenticator-only step-up would mean retyping the password. This deviation is an open owner decision (section 21). Section 15 shows the dialog.

10. Data layer: Drizzle, repositories and row-level security

Schema (`src/db/schema/`): `identity.ts` (users, external_identities), `organization.ts` (organizations, org_units, org_unit_closure, memberships, legacy_teams and the `teams` view), `authorization.ts` (permission_templates, grants, assignments, approval_authorities, emergency_access), `professional.ts` (conversations, conversation_messages, intake_records), `invitations.ts` (invitations, sso_connections, sso_domains), `audit.ts` (audit_events, retention_policies), `content.ts` (experts, credentials, content, channels, polls, leads, event requests), `relations.ts` for `db.query.*`. Since migration `0005` neither `users` nor `memberships` carries a legacy role: what a person may do comes from grants only. Conventions: snake_case columns (`casing: 'snake_case'`), text ids with `gen_random_uuid()::text` defaults, `timestamptz(3)`, `updated_at` maintained by Drizzle.

Row-level security. Tables with an organization column (`org_units`, `memberships`, `grants`, `approval_authorities`, `invitations`, `sso_connections`, `channels`, `event_requests`, `conversations`, `intake_records`) declare a `pgPolicy` whose predicate is `coalesce(current_setting('app.current_org_id', true), '') = '' OR org_id = current_setting('app.current_org_id', true)` (platform-wide grants with a null organization stay visible). Tables that reach their organization only through a join use `tenantPredicateVia()` with a correlated `EXISTS`: `posts` through `channels`, `polls`, `poll_options` and `poll_votes` through `legacy_teams`, `org_unit_closure` through `org_units` (migration `0002_rls_legacy_content_closure.sql`) and `conversation_messages` through `conversations` (migration `0003`). Every one of the sixteen tenant tables has RLS enabled and forced. `emergency_access` is platform-level and has no policy. `DbService.withOrgScope(orgId, fn)` runs `fn` in a transaction after `select set_config('app.current_org_id', $1, true)`; everything inside sees and writes only that organization. `DbService.withTransaction(fn)` is the same without the setting, for platform writes such as platform grants and emergency access. Code that runs without the setting (principal loading, seeds, jobs) is in system context and sees all rows. Policies bind only roles that are neither superuser nor table owner: migrations run as the owner, the API connects as a login that is a member of `nestself_app` (Docker: `nestself_api` created by `docker/initdb/01-app-role.sql`). A local Homebrew superuser is not restricted; `test/db/schema.spec.ts` proves the policies with a non-superuser login.

Repositories (`src/db/repositories/`): `identity`, `authorization` (grants, assignments, delegation walks, emergency access), `approval-authorities`, `professional`, `audit`, `leads`, `invitations`, `memberships`, `org-units`. They are the only place with Drizzle query builders; every method accepts an optional

`tx` so services can compose them inside `withOrgScope`. Services depend on repositories, which is why unit tests mock plain methods and app-level tests override `DbService` with `{ ping, onModuleInit, onModuleDestroy }`.

Migrations. `pnpm --filter @nestself/api db:generate` diffs the schema against `drizzle/meta` and writes a new SQL file; `db:generate -- --custom --name=<name>` creates an empty file for hand-written SQL; `db:migrate` applies pending files (tracked in `drizzle.__drizzle_migrations`); `db:seed` upserts the fixtures. `0001_audit_partitions_rls_roles.sql` holds what drizzle-kit cannot express: monthly range partitions and `audit_events_ensure_partitions()`, the append-only triggers, the `nestself_app` role and its privileges, `FORCE ROW LEVEL SECURITY`. `0002_rls_legacy_content_closure.sql` adds the join-based policies above, forces them, and makes `audit_events_ensure_partitions()` `SECURITY DEFINER` so the application role can create partitions from the nightly job. Never let drizzle-kit alter `audit_events` directly; write a custom migration.

Phase 2 added three migrations. `0003_professional_domain.sql` creates the confidential-domain tables with their policies and forced RLS. `0004_emergency_access_request.sql` makes `emergency_access.approved_by_user_id` nullable and adds `approved_at` and `requested_duration_minutes` (default 240), so a request can wait for its approver. `0005_drop_legacy_roles.sql` drops `users.platform_role`, `memberships.role` and the `org_role` and `platform_role` enum types (D-36); a hand-written `DO $$` guard at its top raises, naming the first user, when a legacy role still lacks its backfilled grant. On a database that still holds legacy roles, run `pnpm --filter @nestself/api db:backfill-grants` before `db:migrate` applies `0005`, from any migration state at or after `0001` (Drizzle's migrator cannot stop at `0004`; it applies every pending migration), then `db:migrate`; if anything is unbackfilled the guard refuses the drop (`apps/api/README.md`, "Legacy role backfill"). A fresh database needs no backfill. `memberships.team_id`, `legacy_teams` and the community tables stay until the community feature is retired, and the contracts keep `OrgRole` and `PlatformRole` for sample mode.

11. Organization and operations administration

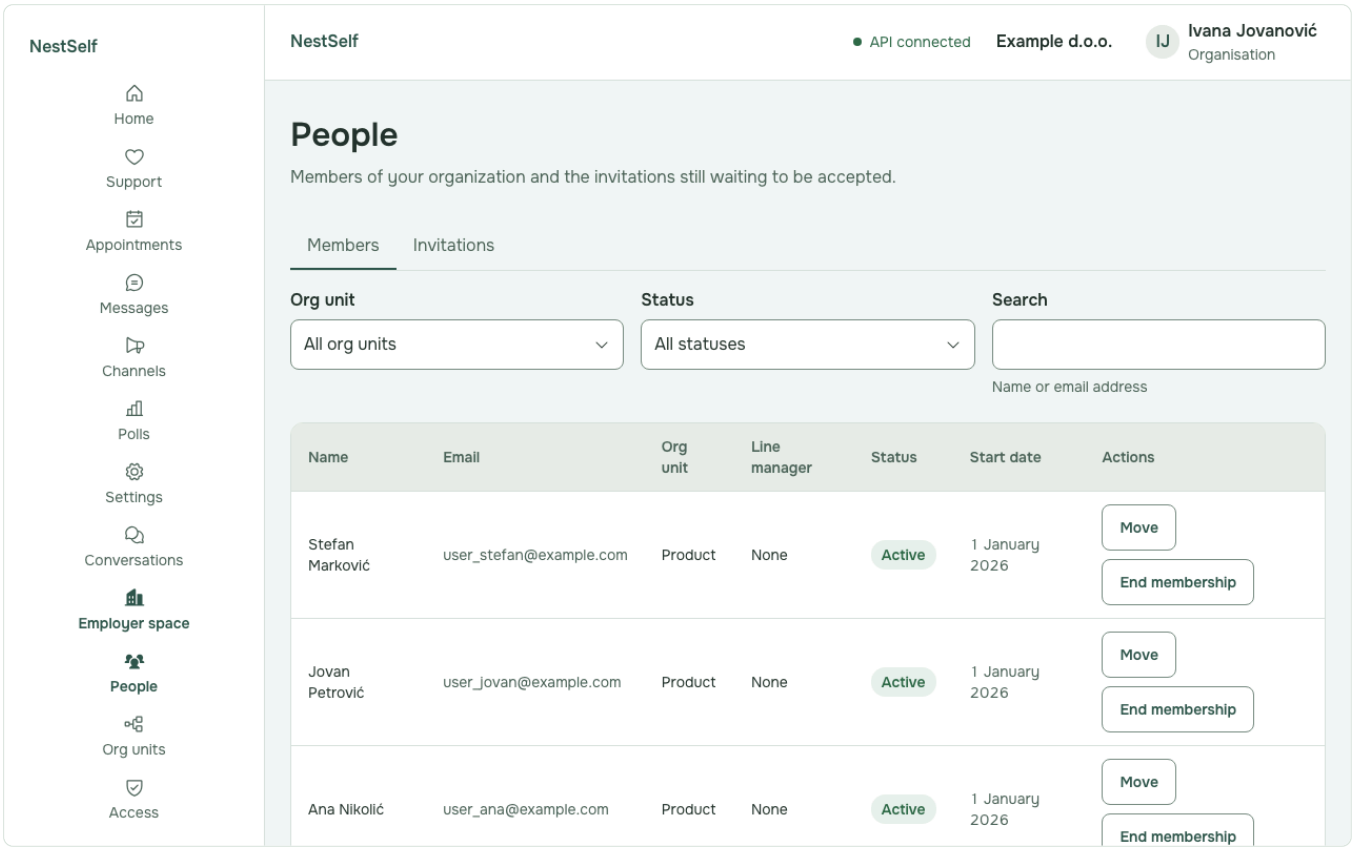
All `/orgs/:orgId/*` routes require the active context to be that organization (`orgResourceFromContext`). All `/ops/*` routes require a web session in the operations context (`operationsResource`, or `operationsOrgResource(type)` when the route names an organization). Writes run inside `withOrgScope` (or `withTransaction` for platform rows) and audit through `AuditWriter`.

Area	Routes and actions	Files
Invitations	`GET /orgs/:orgId/invitations` (pending, cursor), `POST /orgs/:orgId/invitations` and `DELETE .../:invitationId` (`people.invite`), `POST .../import` (`people.import`); public `GET /invitations/:token/preview`, `POST /invitations/accept`	`src/invitations/` (`invitations.service.ts`, `invitation-activation.service.ts`, `invitation-token.ts`, `invitation-mailer.ts`, `ses-invitation-mailer.ts`)
Invitations for assigned clients	`GET /ops/orgs/:orgId/invitations`, `POST` and `DELETE .../:invitationId` (`people.onboard`), `POST .../import` (`people.import`)	`src/invitations/ops-invitations.controller.ts`
Organizations, operations side	`GET /ops/organizations` (`client.configure` at platform scope; cursor, name or slug prefix search), `GET /ops/organizations/:orgId` (`client.configure` through `operationsOrgResource`, so a `client_operations` user opens an assigned client)	`src/organizations/`
Org units	`GET /orgs/:orgId/org-units` (`program.overview`), `POST`, `PATCH .../:unitId`, `POST .../:unitId/move` (`org_units.manage`)	`src/org-units/` (`closure-moves.ts` holds the pure closure-table algebra)
Members	`GET /orgs/:orgId/members` (`program.overview`; cursor pagination, filters), `PATCH .../:membershipId` (`people.move`), `POST .../:membershipId/end` (`people.deactivate`)	`src/members/`
Members of assigned clients	`GET /ops/orgs/:orgId/members` (`client.configure`), `POST .../:membershipId/end` (`people.deactivate`)	`src/members/ops-members.controller.ts`
Grants	`GET /orgs/:orgId/grants` (`access.preview`; `?subjectUserId=&includeRevoked=`), `GET .../preview?userId=` (`access.preview`), `POST` and `PATCH .../:grantId` (`access.grant`), `POST .../:grantId/revoke` (`access.revoke`), `POST .../delegate` (`access.delegate`), `POST .../transfer-ownership` (`access.grant`, caller must hold the owner grant)	`src/grants/`
Approval authorities	`GET /orgs/:orgId/approval-authorities` (`access.preview`), `POST` and `PATCH .../:id` (`access.grant`)	`src/grants/approval-authorities.*`
Grants, operations side	`GET` and `POST /ops/grants` (platform grants; `access.preview`, `access.grant`), `POST /ops/grants/:grantId/revoke` (`access.revoke`); `GET /ops/orgs/:orgId/grants`, `GET .../preview`, `POST`, `POST .../:grantId/revoke`, `POST .../transfer-ownership` (`access.grant`); `GET /ops/users/:userId/access` (`access.preview`)	`src/grants/ops-grants.controller.ts`
Users, operations side	`POST /ops/users/:userId/suspend`, `POST .../reinstate` (`people.deactivate`)	`src/users/`
Confidential domain	`GET /professional/conversations` (the caller's assigned conversations) and `GET /professional/me/conversations` (the participant's), metadata only, cursor; `GET /professional/conversations/:id` and `GET /professional/intake/:id` (`conversation.read.assigned`), `GET /professional/me/conversations/:id` (`conversation.participate`); `@Authenticated()`, authorized in the service (the lists run the engine per row, without audit or step-up)	`src/professional/`
Emergency access	`GET /ops/emergency-access` (`audit.read`, optional `?userId=`), `POST` (`access.emergency.request`), `POST .../:id/approve`, `.../:id/deny`, `.../:id/revoke` (`access.emergency.approve`), `GET .../:id/records/:recordType/:recordId` (`@Authenticated()`, `records.read.emergency` in the service)	`src/emergency/`

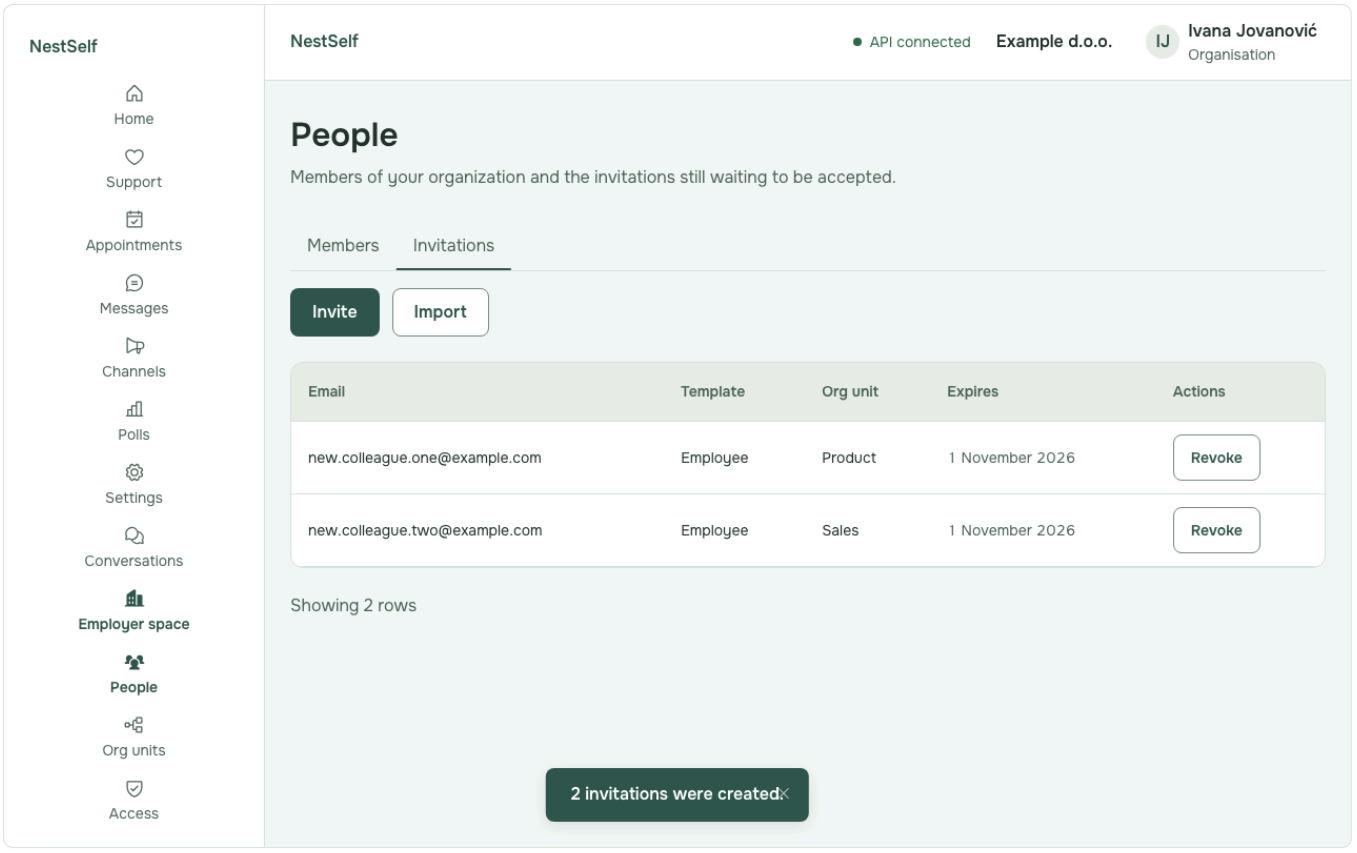
The routes for assigned clients use different actions from their `/orgs` twins because the `client_operations` template holds `people.onboard` and `client.configure`, while `people.invite` and `program.overview` are HR actions (strategy 4.4). The ops invitation routes validate bodies with `ZodValidationPipe` and the `/orgs` invitation routes with a local helper, so their 400 bodies differ slightly.

Invitation tokens are 32 random bytes; only the SHA-256 hash is stored, with a 7-day expiry. An invitation carries only an active template of the `organization` or `employee` workspace (`findInvitableTemplate` in `src/grants/grant-rules.ts`): create answers 400 `validation` with `details.templateKey` for an operations or expert template, import lists such rows in `errors` and creates the rest, and acceptance refuses a stored invitation with another template as `invalid_invitation` , so an invitation can never mint a staff or platform grant. In development the create response adds `devToken` and the logging mailer prints the link; with `SES_FROM_EMAIL` set the SES mailer sends it. Membership end sets `ENDED` , revokes the user's grants scoped to the organization and the whole delegation tree below them (`revokeGrantTree` : roots record `membership_ended` , descendants `parent_revoked`), audits first, invalidates every delegatee, then calls `revokeAllAccess` (SESS-R-22, SESS-R-23). Suspension sets `users.suspended_at` , disables the identity-provider user, audits `user.suspended` (SECURITY tier) and revokes all access; `authorize()` denies suspended users on the next request anyway.

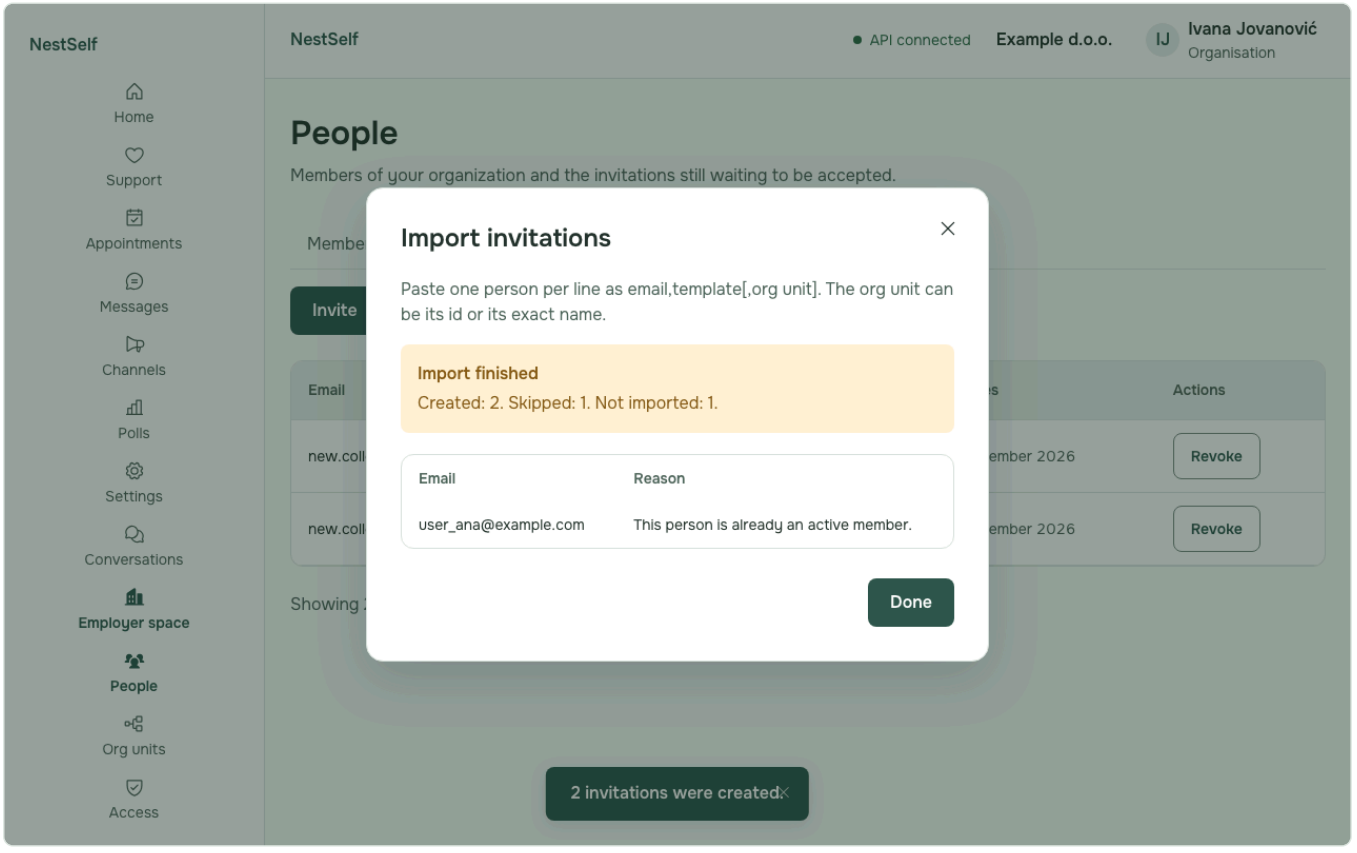
People and org units in the workspace. `/hr/people` (`features/people/`) has two tabs. *Members* lists the organization's members page by page with the API cursor ("Load more", never capped), filtered by org unit, status and a debounced name or email search, with the org unit and the line manager by name. Move (`people.move`) changes the org unit and the line manager and sends only what changed; End membership (`people.deactivate` , administration step-up) asks for a reason and is not offered on one's own row or on ended members. *Invitations* lists the pending invitations; Invite takes an email address, an invitable template (active `organization` and `employee` templates, the same rule as the server) and an optional org unit; Import takes pasted `email,template[,org unit]` lines (unit by id or exact name, up to 500 rows), shows the problems line by line and sends nothing until they are fixed, then shows how many were created and a table of the refused rows with translated reasons; Revoke ends a pending invitation. `/hr/org-units` (`features/org-units/` , `org_units.manage`) shows the tree indented by depth with Rename, Add child (department or team) and Move (to any unit that is not itself, below itself or its current parent; the company root does not move).



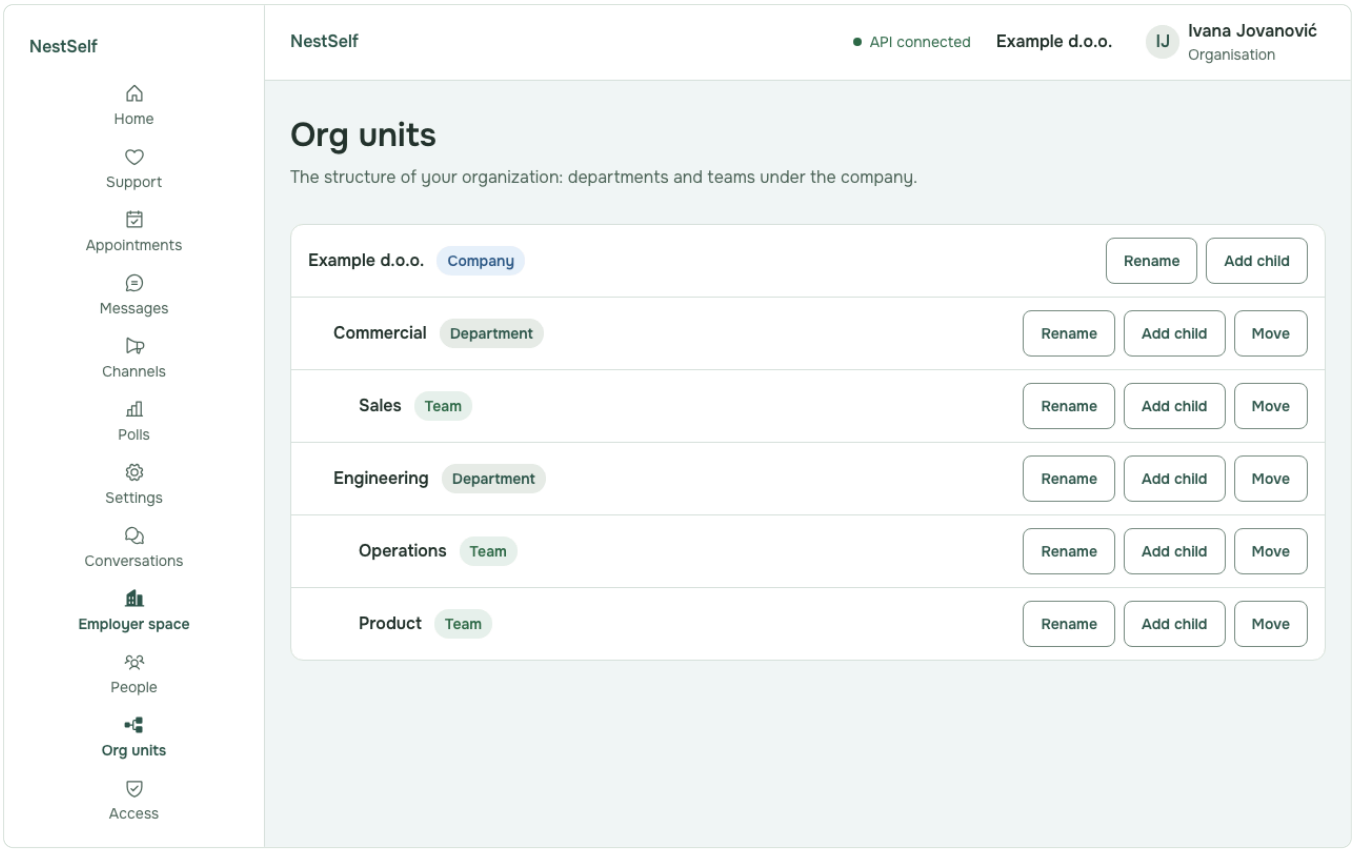
`/hr/people` , Members: filters, the member table with org unit, line manager and status, Move and End membership per row.



The Invitations tab with two pending invitations, created by the import below in the disposable e2e database.



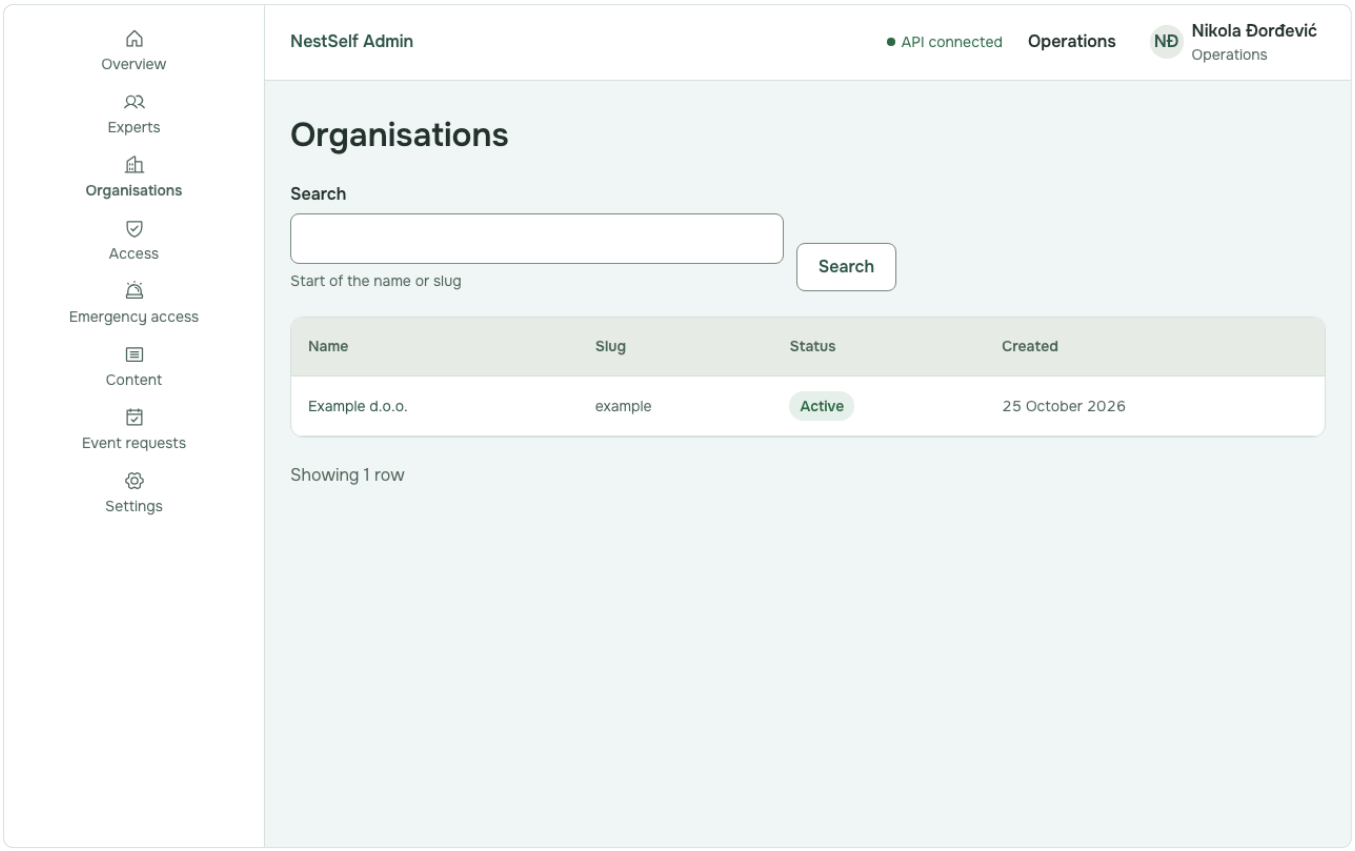
Import result: two invitations created; the row for Ana Nikolić is refused because she is already an active member. The API counts a refused duplicate or member row both as skipped and as not imported (section 21).



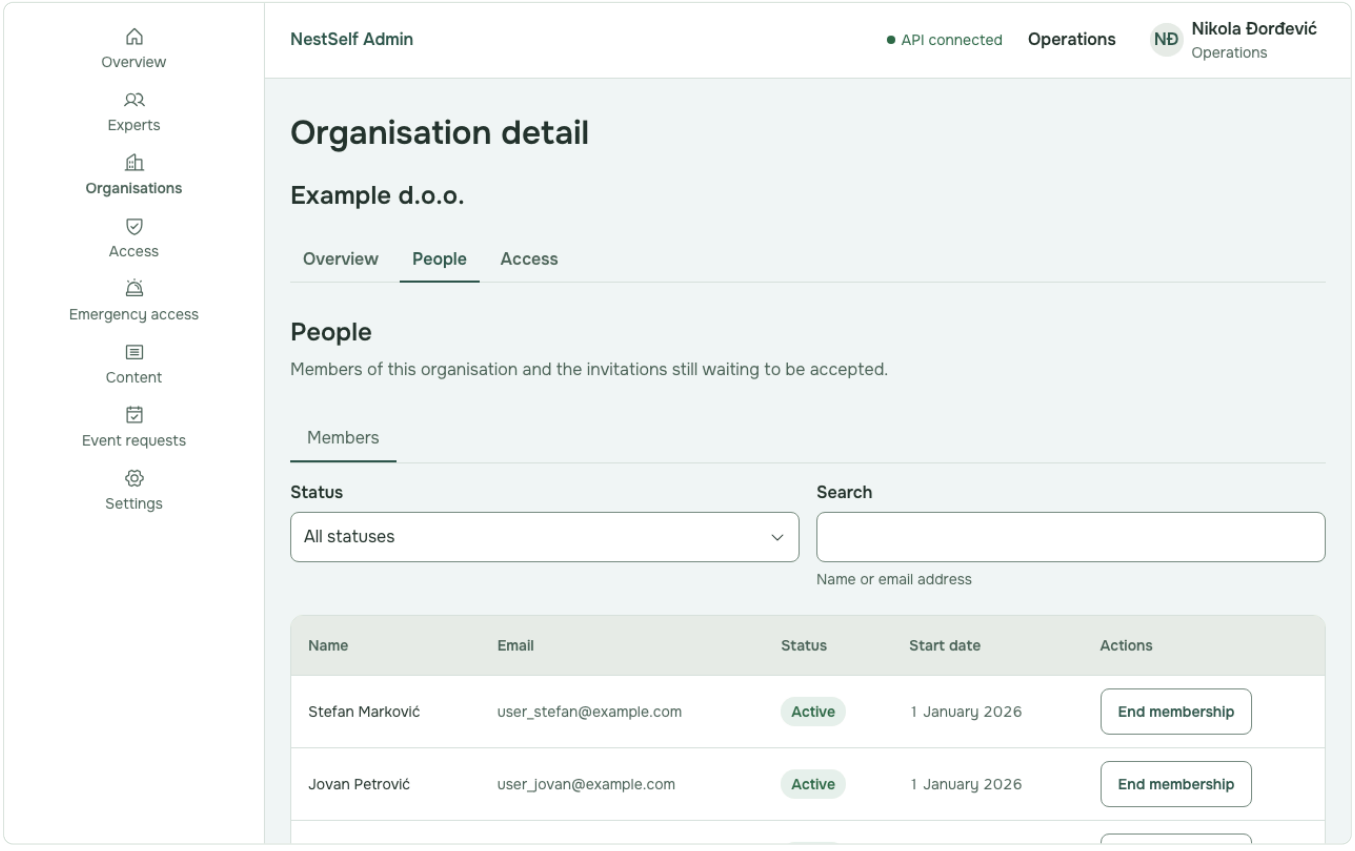
`/hr/org-units`: the sample tree under the company, Example d.o.o.

`PATCH /orgs/:orgId/members/:id` returns the full member item, the member's name and email included, since 2026-10-25. It used to answer with empty strings, which the client's `MemberItemSchema` rejects, so every successful Move showed "Something went wrong"; the Playwright journey `hr-people.spec.ts` found it.

Organizations and people in the admin app. `/organizations` (`features/organizations/`) lists organizations with a prefix search and "Load more"; a caller without the platform-scoped list (a `client_operations` user) sees a note that access is limited to assigned clients and opens them by link. The detail page has the tabs Overview (id, slug, status, the owner from the active `employer_owner` grant and Transfer ownership through the staff route, also when there is no owner yet), People and Access (section 11.1). People (`client.configure`) lists the members with a status filter and search and offers End membership with a reason (`SUSPENDED_BY_PLATFORM` among them); its Invitations tab (`people.onboard`, import with `people.import`) mirrors the workspace one at organization level, because the ops API has no org-unit routes. `platform_owner` holds no `people.onboard`, so a platform owner sees Members only.

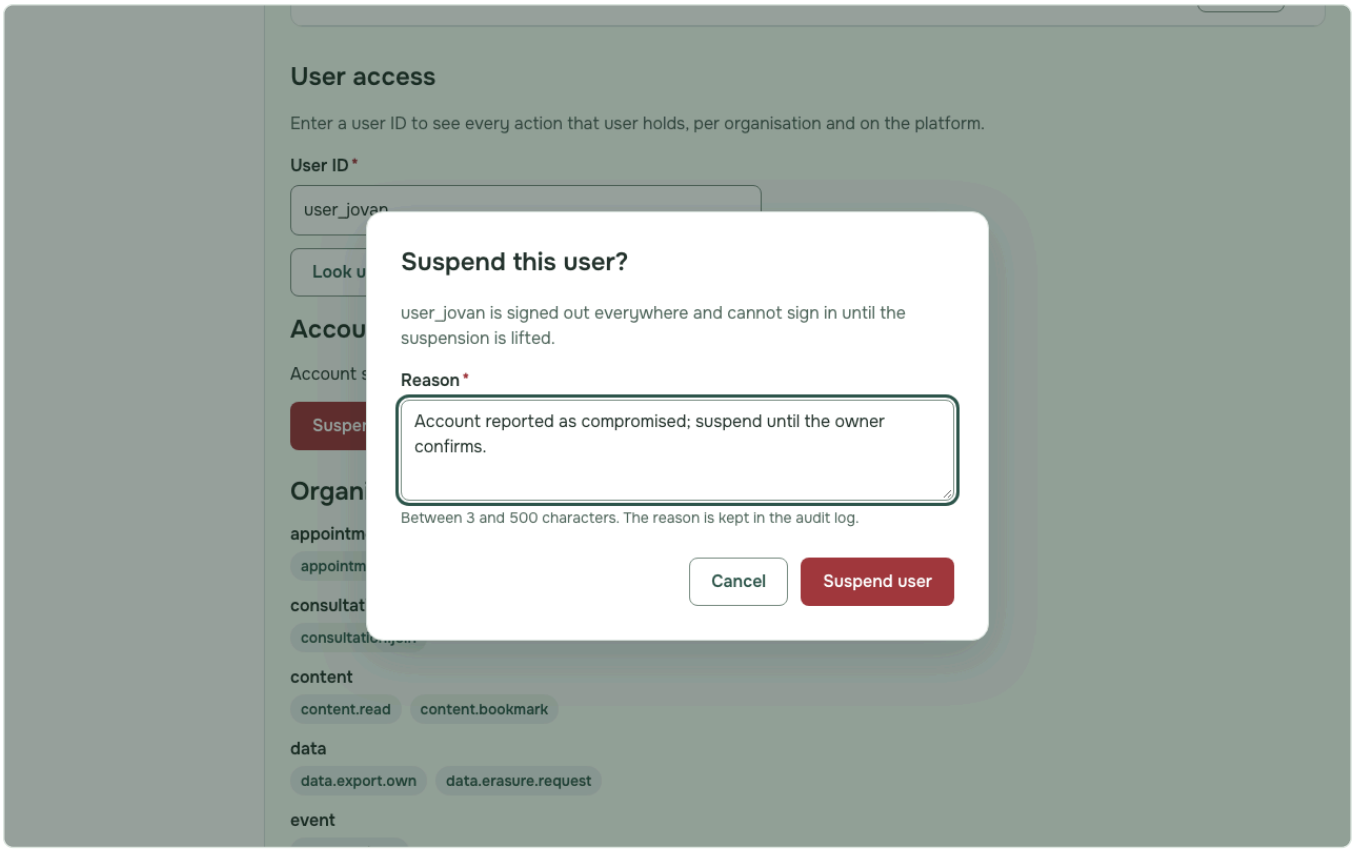


Admin `/organizations` for a platform owner: the sample organization with its slug, status and creation date.



An organization's People tab in the admin app: members with End membership.

Suspend and reinstate. Once the user-access lookup (`/access` , and the Access tab of an organization) returns, `features/users/user-account-actions.tsx` shows the account state and Suspend user (reason of 3 to 500 characters) or Reinstate user, both `people.deactivate` and through `withStepUp` . For one's own account Suspend is replaced by a note, and the API refuses it too with 409 `cannot_suspend_self` . Reinstatement now invalidates the cached principal, so a reinstated user can sign in at once.

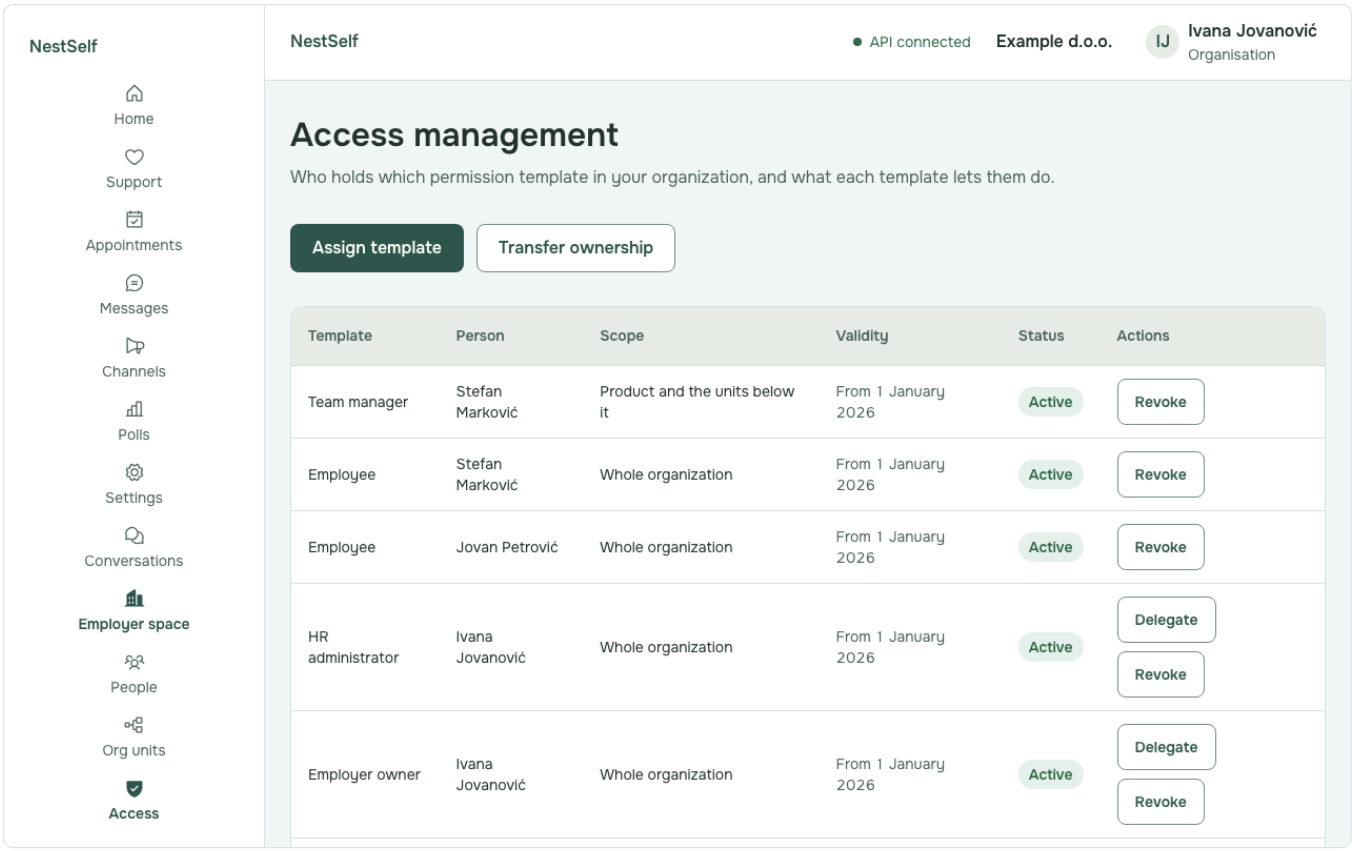


Suspend this user, opened from the lookup of `user_jovan` and not confirmed.

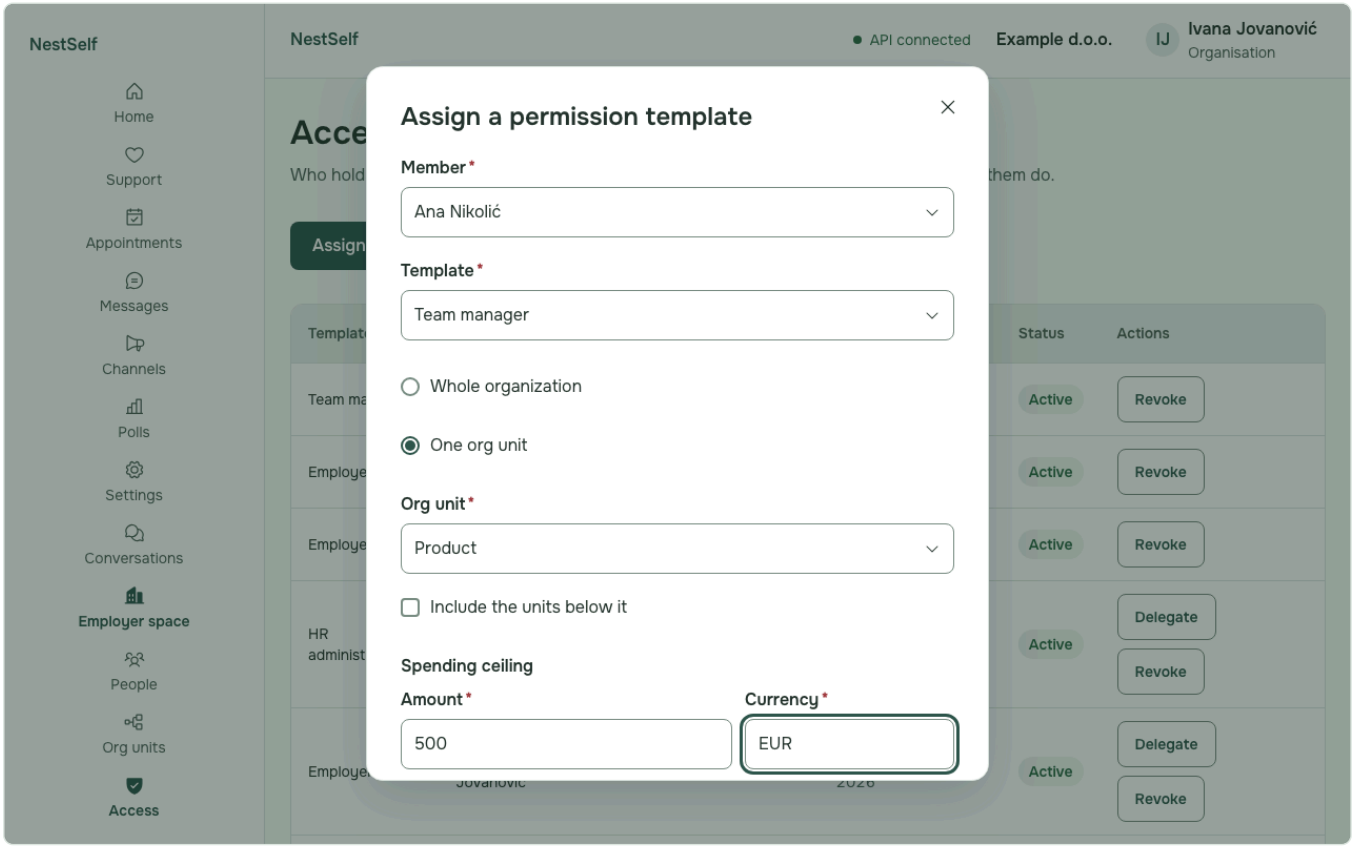
`people.deactivate` is held by three templates: `hr_admin` (organization scope, ends memberships of its own organization through `/orgs/:orgId/members/:id/end`), `platform_owner` (platform scope, the only template that reaches `/ops/users/:id/suspend`, whose resource has no organization) and `client_operations` (scope `assigned_clients`, through `/ops/orgs/:orgId/members/:id/end` for an organization the caller holds a CLIENT assignment to). The sample data has no platform owner; tests create that grant from the template.

11.1 Grant administration

Create (`POST /orgs/:orgId/grants`, `{ templateKey, subjectUserId, scopeType, scopeOrgUnitId?, includeDescendants?, conditions, validFrom?, validUntil? }`). The template must be active (400 `template_inactive`) and belong to the `organization` workspace (400 `validation`). The scope is `organization` or `org_unit`, with a unit of this organization (404 otherwise). The subject must hold an ACTIVE membership (409 `subject_not_member`). Every `requiredConditionTypes` entry of the template must be present (400 `missing_conditions`, `details.missing`). Actions are copied from the template, `validFrom` defaults to now and `validUntil` must come after it. A second active `employer_owner` grant is refused with 409 `owner_exists` (see "Organization ownership" below).



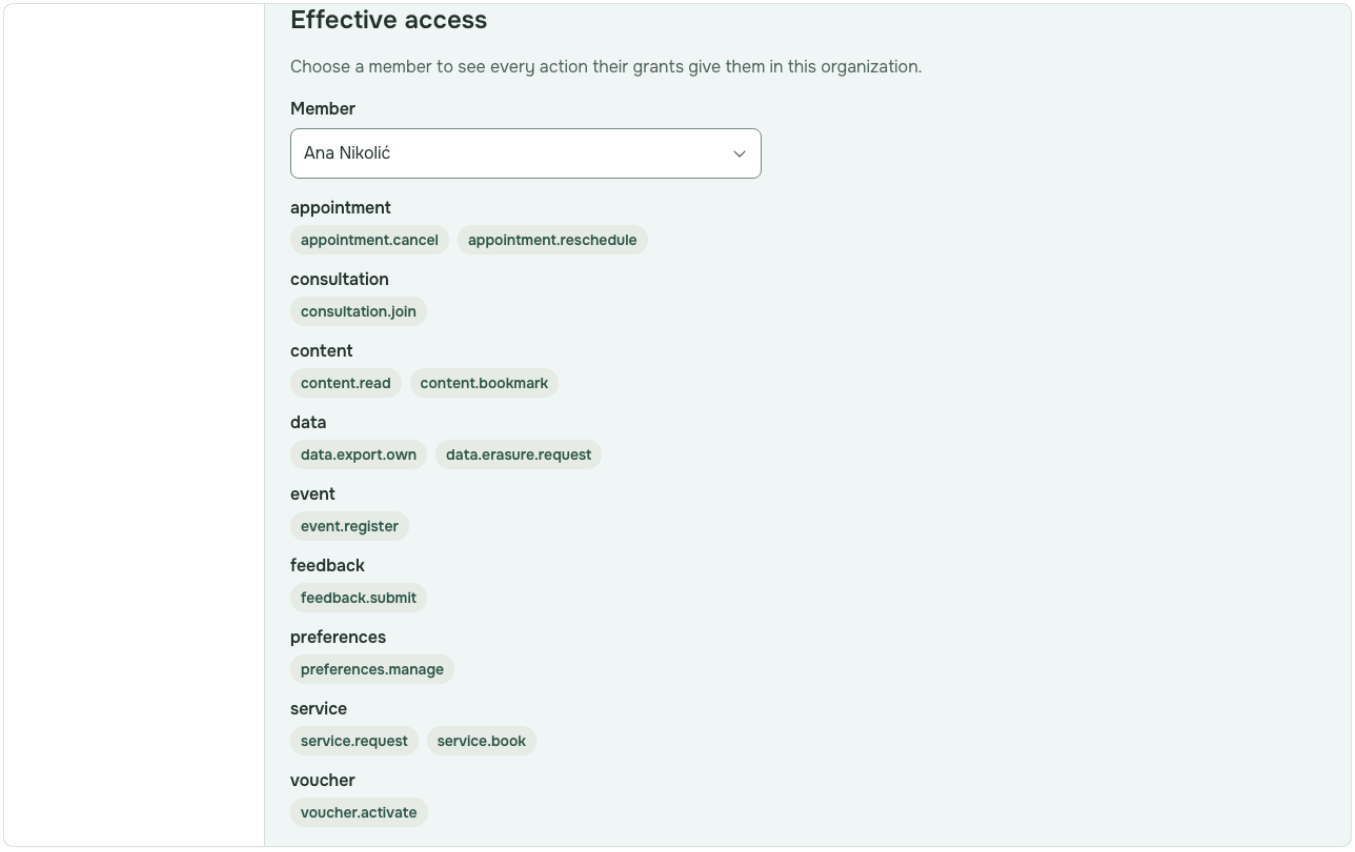
Workspace `/hr/access` for Ivana Jovanović, the sample HR administrator and employer owner: the organization's grants with Delegate and Revoke per row, and Assign template and Transfer ownership above the table.



Assign: a `team_manager` grant for Ana Nikolić on the Product unit, with the spending ceiling the template requires.

Update (`PATCH`) changes conditions and the validity window only. A revoked grant answers 409 `conflict` , another organization's grant 404. When the patch changes conditions and the grant has active delegated children, each child is checked against the patched parent; if any falls outside, the update is refused with 409 `conflict` and `details.children` naming them, so the administrator adjusts or revokes those first. A shorter validity window is not checked against the children: the engine denies them at runtime once the parent has expired.

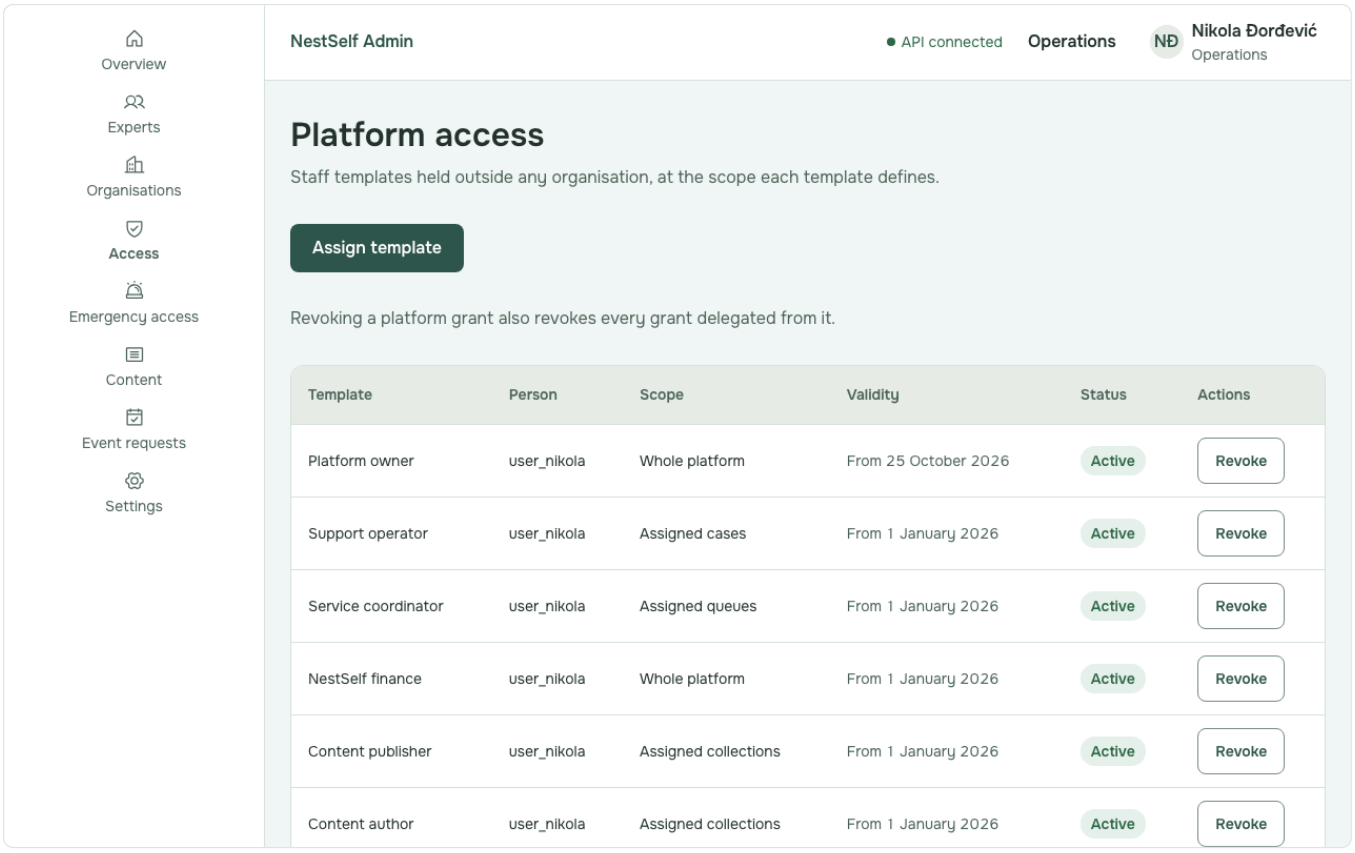
Revoke takes a reason (3 to 500 characters) and revokes the grant and every descendant through `revokeGrantTree`; an already revoked grant answers 409. **Preview** (`?userId=`) requires the subject to be an active member (404 otherwise), runs `effectiveAccess` for the subject in this organization and drops the confidential actions (AUDIT-R-02).



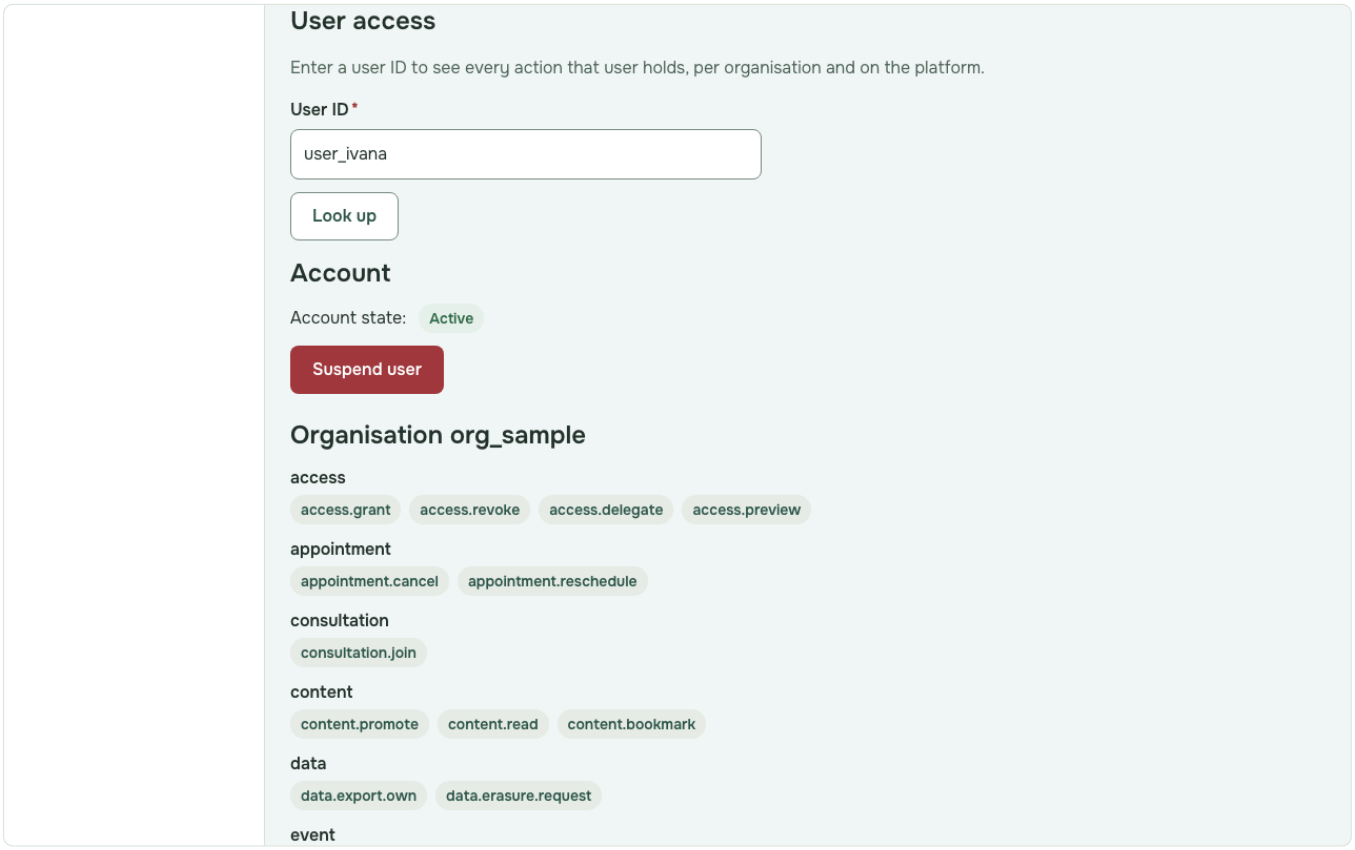
The effective-access panel of `/hr/access`: every action Ana Nikolić's grants give her in the organization, grouped by prefix.

Every write records one audit event (`grant.created`, `grant.adjusted`, `grant.revoked`, `grant.delegation_revoked` per descendant, `grant.delegated`) with field names, counts and condition types, never values, and invalidates the subject and every delegatee.

Operations side. `POST /ops/orgs/:orgId/grants`, its list, preview and revoke use the same service. `POST /ops/grants` creates a platform grant (`scope_org_id` null) from an `operations` workspace template at the template's own scope type, inside `DbService.withTransaction` with its audit call. `POST /ops/grants/:grantId/revoke` (reason body, write throttle) revokes a platform grant and its descendants: 404 `not_found` for a missing or organization-scoped grant (those go through `/ops/orgs/:orgId/grants/:grantId/revoke`), 409 `conflict` when already revoked; otherwise `revokeGrantTree`, `grant.revoked` and one `grant.delegation_revoked` per descendant in one system transaction, then the subject is invalidated for the platform, their membership organizations and their CLIENT organizations (the same set as on create) and every delegatee below. This is how a member of staff is off-boarded from the operations workspace. `GET /ops/users/:userId/access` returns one context per active membership plus a platform context, each with actions (confidential ones dropped), grant ids and assignment ids. The ops side has no org-unit, update, delegation or approval-authority routes.



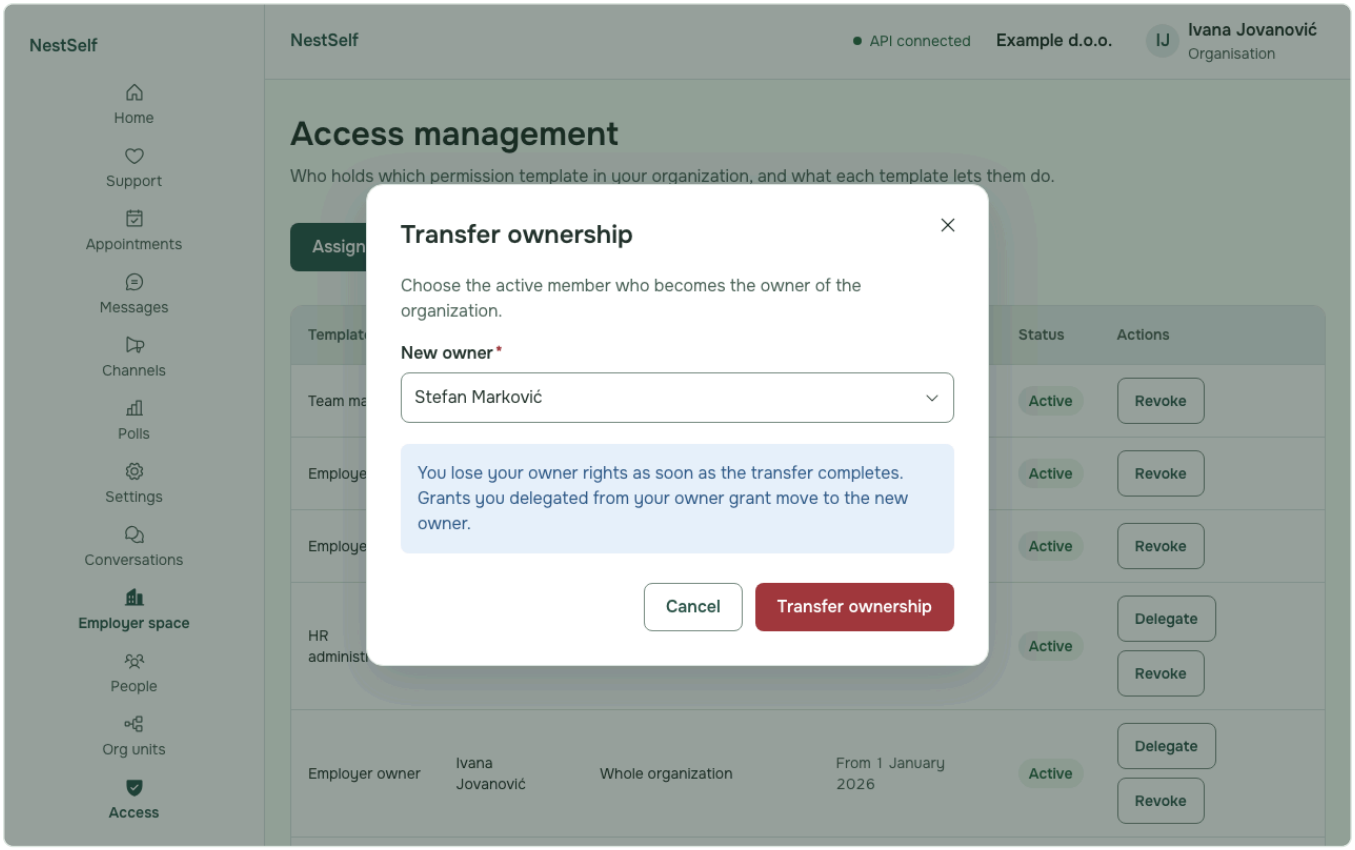
Admin `/access`: platform grants with revoke. The first row is the `platform_owner` grant the screenshot run gives Nikola Đorđević; the sample data has none (section 19).



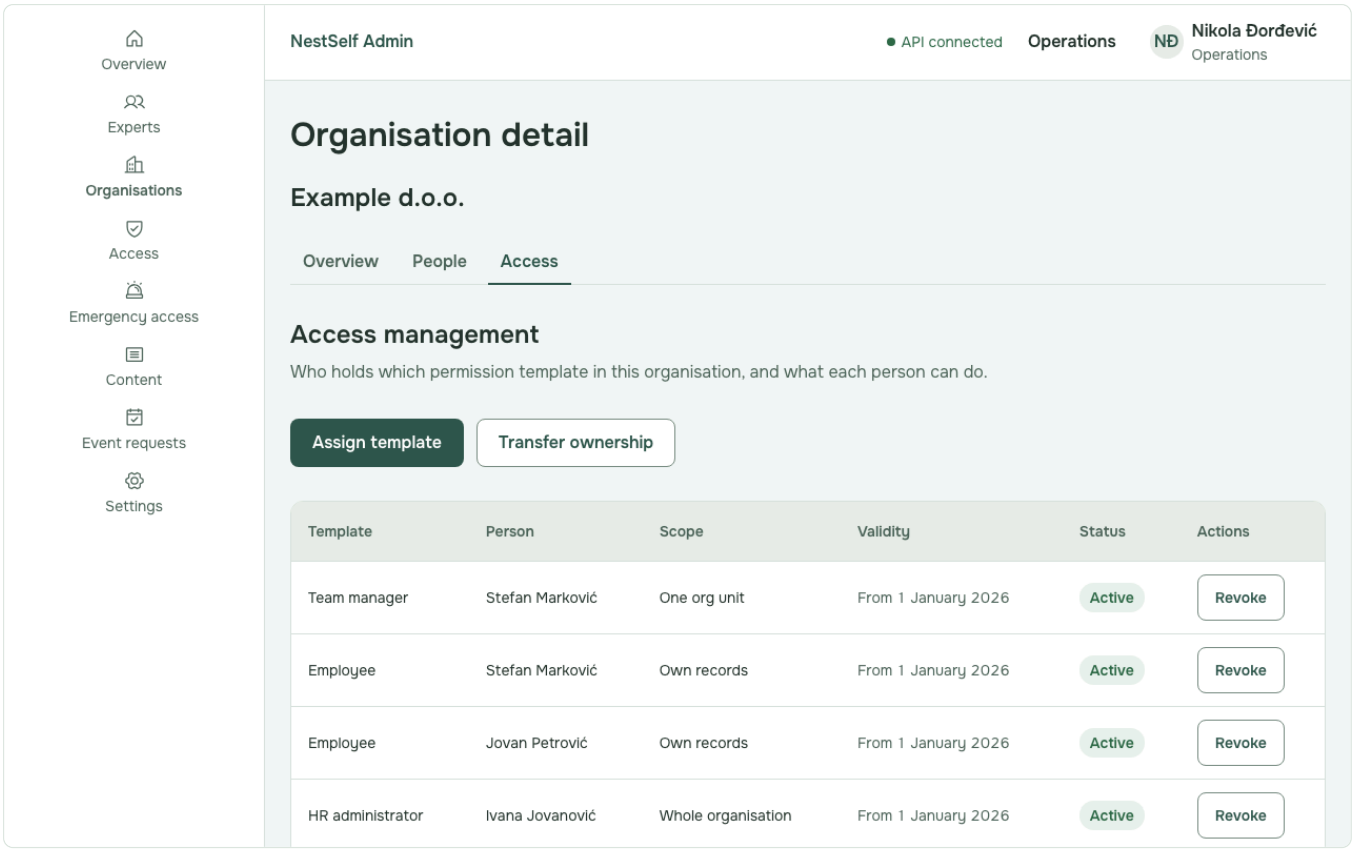
The user-access lookup (`GET /ops/users/:userId/access`) for `user_ivana`: one section per context.

Organization ownership (owner model, product owner decision of 2026-10-24; strategy 4.4 amendment). `employer_owner` is the single superadmin of an organization. It holds `access.grant`, `access.revoke`, `access.delegate`, `access.preview`, `policy.manage`, `service.configure` and `org.settings`, and its three `access.*` writes are in the `administration` step-up class. `hr_admin` holds none of the `access.*` actions; the owner gives an HR administrator grant and revoke rights by assigning or delegating them. NestSelf staff with `platform_owner` keep every right through the `/ops/orgs/:orgId/*` routes, so support can always change anything for an organization.

- **One owner per organization.** `GrantsService` keeps at most one active (unrevoked, not expired) `employer_owner` grant per `scopeOrgId`. `AuthorizationRepository.findActiveOwnerGrants` takes a transaction-scoped advisory lock on the organization's ownership and returns the active owner grants; create (both route families), a PATCH of an owner grant's window, the transfers, and invitation create, import and acceptance refuse a second owner with 409 `owner_exists` and `details.currentOwnerUserId`. A grant whose window starts later already counts, so a future-dated second owner is refused too, and a PATCH cannot revive an expired owner grant next to the current one.
- **Transfer by the owner**: `POST /orgs/:orgId/grants/transfer-ownership` with `{ newOwnerUserId }` (`access.grant`, so step-up applies). The caller must hold the started owner grant (403 `forbidden` otherwise), the new owner must be an ACTIVE member (409 `subject_not_member`) and must not be the current owner (409 `conflict`).
- **Transfer by staff**: `POST /ops/orgs/:orgId/grants/transfer-ownership` (`access.grant` through `operationsOrgResource`) has no current-owner requirement and creates the first owner when the organization has none.
- **One transaction** (`transferOwnership`, `withOrgScope`): revoke the old owner grant only (reason `ownership_transferred`), create the new one from the template, and re-parent the old grant's active children to the new grant (`AuthorizationRepository.reparentActiveChildren`). The bounds still hold because the new grant has the same template and actions; without the move the engine would deny the children (revoked parent) and the nightly sweep would end them. Audit: `grant.revoked` and `grant.created` with `details.transfer = true` (plus `byStaff = true` on the ops route), and one `grant.adjusted` per moved child with `details.reparented = true` and `fromGrantId`. The response is `OwnershipTransferResponse` (`revokedGrantId`, null when staff created the first owner, and `createdGrant`). The old and new owners, the moved children's subjects and the delegatees below both grants are invalidated afterwards.
- The seed makes the sample HR admin, Ivana Jovanović, the owner of `org_sample` (`grant_user_ivana_employer_owner`), which is what opens `/hr/access` in development and in `pnpm e2e:auth`.



Transfer ownership in the workspace, shown only to the current owner: the new owner is chosen from the active members, and the note says what happens to the owner's rights and delegations.



The admin organization Access tab: staff assign organization-wide templates, revoke and transfer ownership for the organization.

Delegation (`POST /orgs/:orgId/grants/delegate`, `{ parentGrantId, subjectUserId, actions, scopeType?, scopeOrgUnitId?, includeDescendants?, conditions }`). The checks run in this order, and each failure answers 403 `delegation_exceeds_boundary` with a `details.reason` unless noted:

1. the parent grant belongs to this organization (404 otherwise);
2. the parent's subject is the caller (`not_parent_subject`);
3. the scope is `organization` or `org_unit` inside the organization (400 or 404);
4. the delegatee is an active member (`subject_not_member`);
5. the new depth is at most 3 (`delegation_depth_exceeded`);
6. the parent grant bounds the child: actions are a subset, the scope sits within the parent's, a spending ceiling is not higher and categories are a subset (`actions_exceed_parent`, `scope_exceeds_parent`, `spending_ceiling_exceeds_parent`, `categories_exceed_parent`; `grant-rules.ts`);
7. `validateDelegation` over the parent chain loaded from the database, with the parent's `delegationEnvelope`, when present, as an extra restriction.

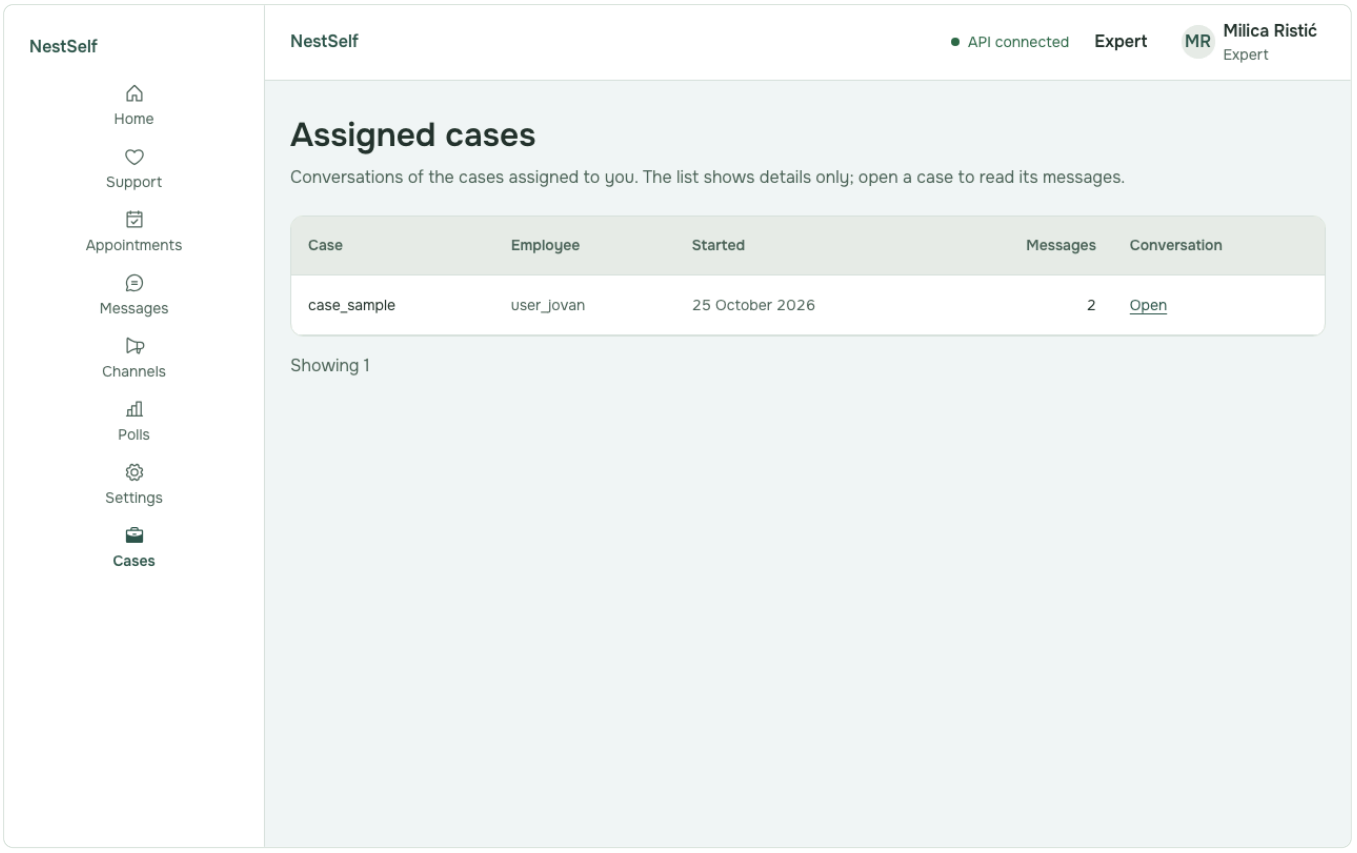
The delegated grant is stored with `templateKey: null`. Strategy AUTHZ-R-13 says a parent without an envelope may not be delegated from at all; here the parent grant itself is the bound in that case (AUTHZ-R-37), which is the reading the plan took. At runtime the delegatee's context carries the ancestor chain (section 9), so the delegated actions apply to them, and the revocation or expiry of any ancestor ends them at once, before a job runs.

Approval authorities. The approver and the alternate must both be active members (409 `subject_not_member`), the alternate must differ from the approver (400 `validation`) and the org unit must belong to the organization. A PATCH never changes `userId`, `orgId`, `orgUnitId` or `delegatedFromId`; an empty PATCH writes nothing. Each write audits `approval_authority.changed` with `details.changedFields`.

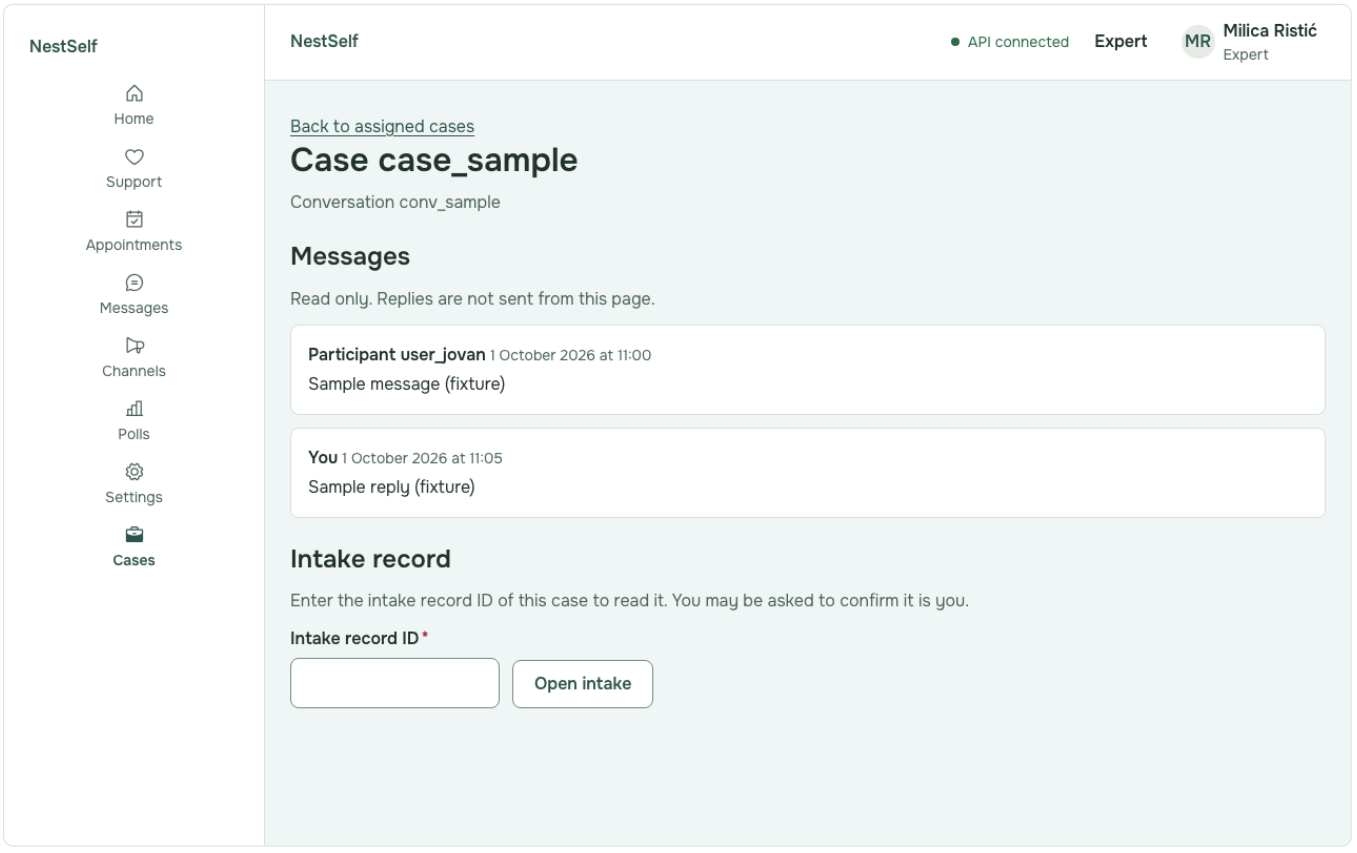
11.2 Confidential domain

`conversations`, `conversation_messages` and `intake_records` (`src/db/schema/professional.ts`, migration `0003`) are placeholder tables with the minimum columns, under forced RLS. No route writes them yet; the seed adds `conv_sample` and `intake_sample` for `case_sample` (`SAMPLE_CASE`) with a CASE assignment for Milica. The service loads the row (404 when it is missing), calls `AuthorizationService.require()` with `{ type, id: caseId, orgId, ownerId: employeeUserId, isConfidential: true }` and only then reads the messages. Two paths are allowed: the participant through `employee` (`own`) on `/professional/me/conversations/:id`, and the assigned expert through the `expert` template (`assigned_cases`) together with a CASE or SUPERVISION assignment whose `targetId` is the case id. The `expert` template gained `conversation.read.assigned` for that path on 2026-10-24. Organization, org-unit, platform and assigned-client grants are hard-denied (golden rows `CONF-DENY-01` to `06`). Bodies are never logged or audited.

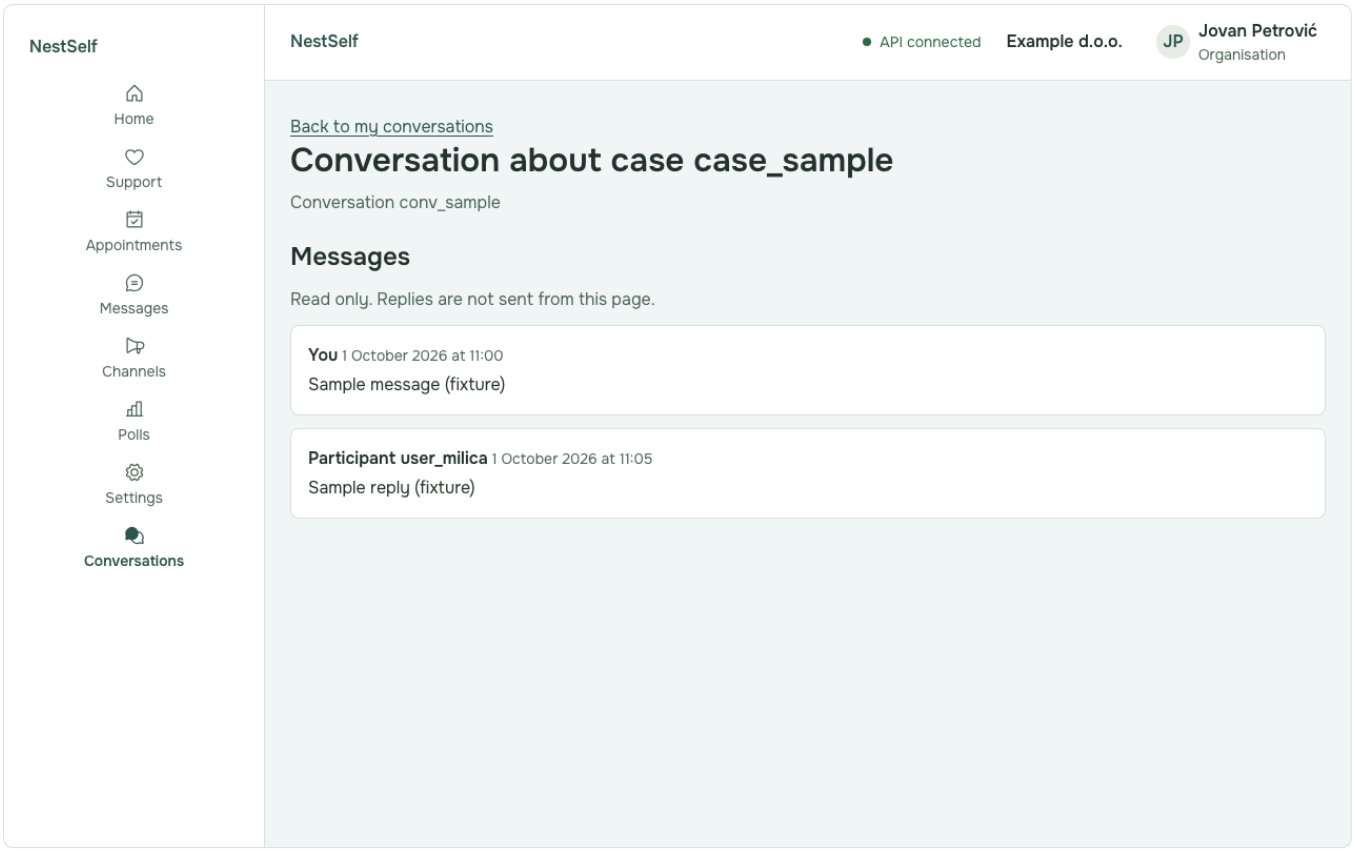
Expert area and participant view. In the workspace, `/cases` (`features/cases/`, route gate `principal.expert` or `conversation.read.assigned`) lists the conversations of the caller's assigned cases (case, employee id, start, message count) with "Load more", and switches an expert to the expert context once. `/cases/:conversationId` reads the conversation through `withStepUp(getConversation)` (the `restricted` class, 15 minutes) and shows the messages read-only, oldest first, with the viewer labelled "You"; a cancelled step-up shows "Nothing was opened" with Try again, and 403, 404 and 429 show copy. The intake section takes the intake record id, because neither the conversation nor the case points to its intake yet (section 21). Employees reach their own conversations under `/conversations` (`conversation.participate`, outside the expert context). Restricted reads are never cached (`gcTime: 0`, no refetch) and never logged. Participants appear as user ids, because the responses carry no names.



`/cases` for Milica Ristić, the sample expert: the conversation of `case_sample`.



The case conversation, read-only, with the intake lookup below. The bodies are the seed's fixture placeholders.



The same conversation as its participant, Jovan Petrović, sees it under `/conversations`.

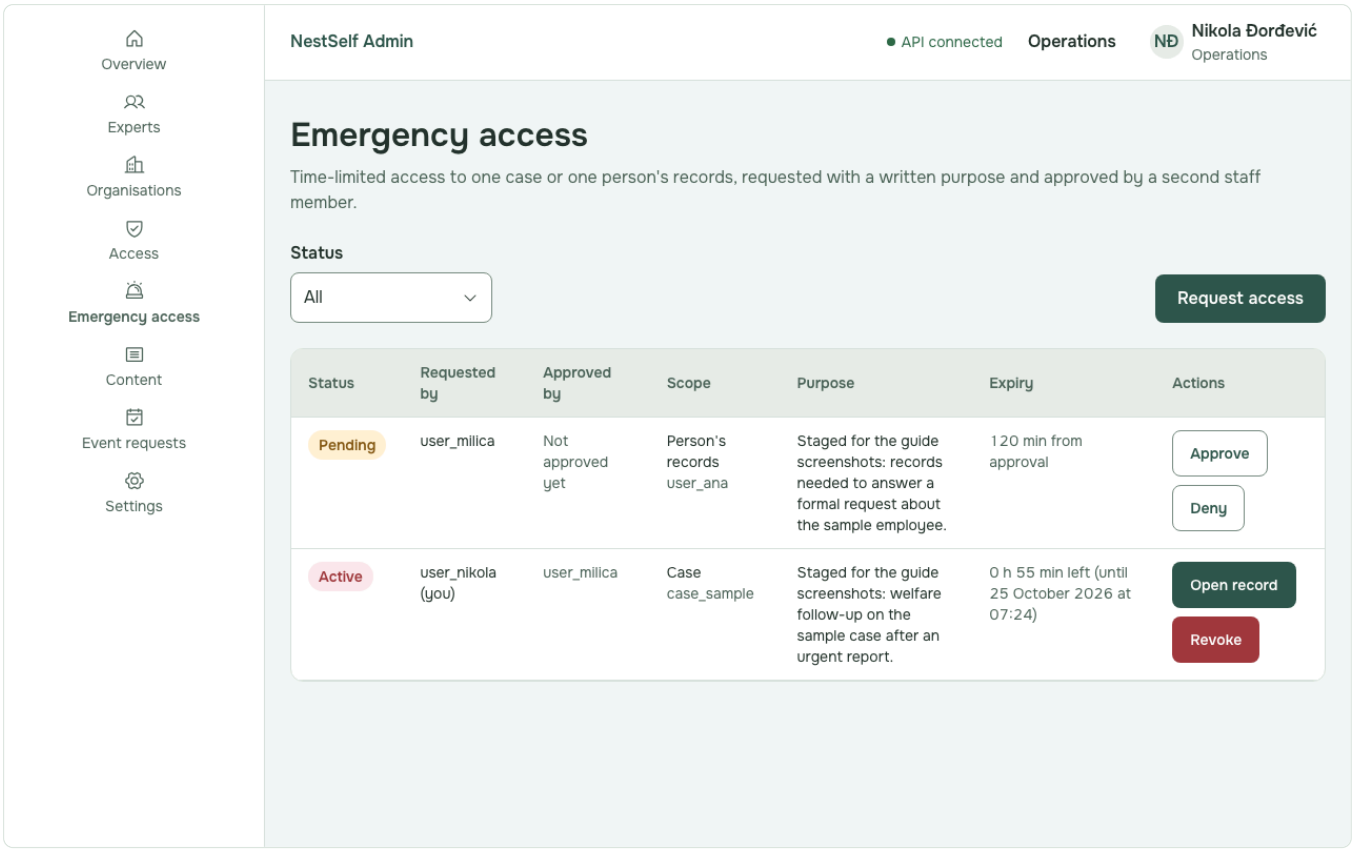
`case_id` is free text with no organization binding, and the assignment match compares `targetId` only. That is harmless while nothing writes these tables; before any write path lands, add a `cases(id, org_id)` table with foreign keys (section 21).

11.3 Emergency access

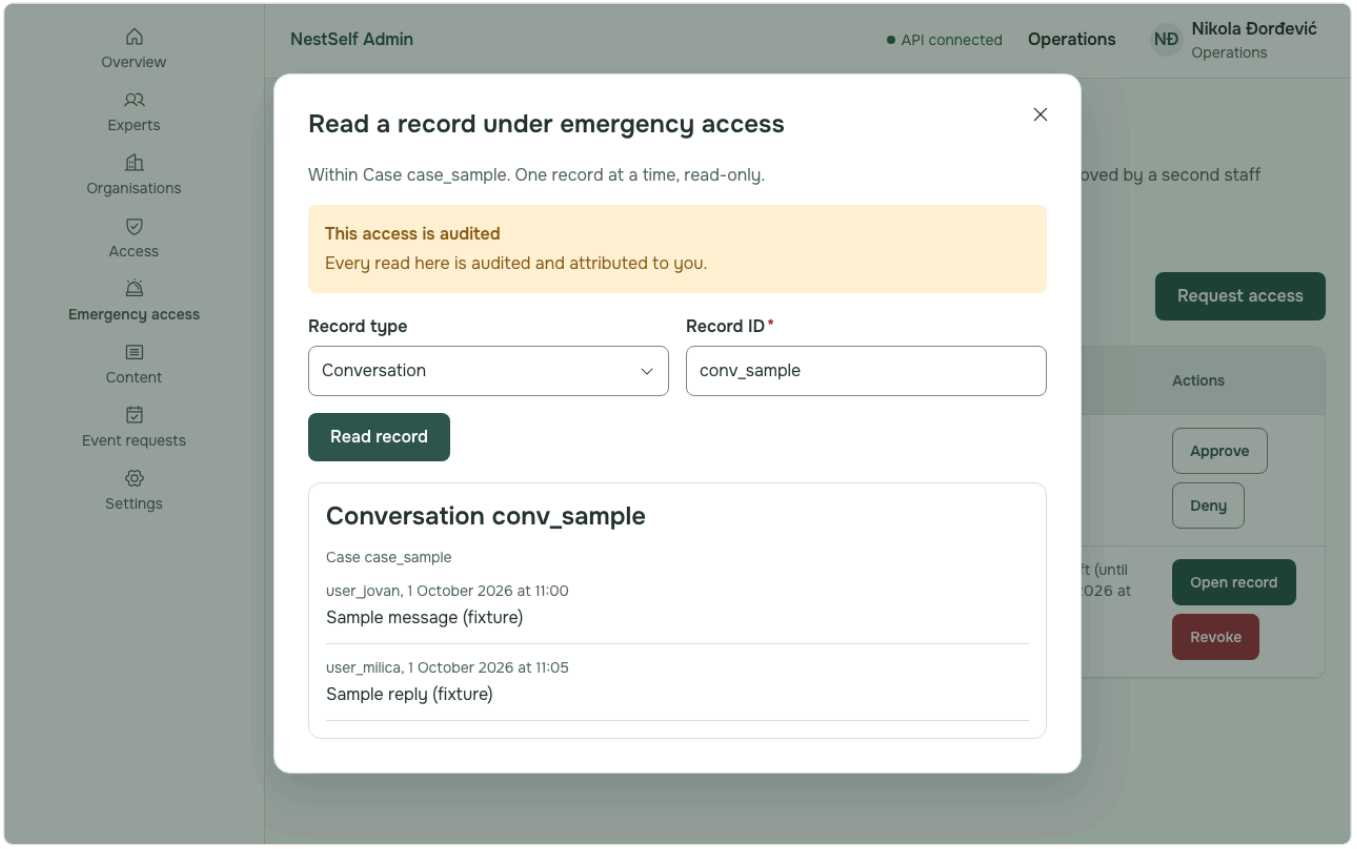
- **Request**: `POST /ops/emergency-access` with `{ purpose (10 to 1000 characters), scopeKind: 'CASE' | 'USER_RECORDS', scopeId, durationMinutes (15 to 240, default 240) }` creates a pending row (`approvedByUserId` null) and audits `emergency_access.requested` with the purpose truncated to 500 characters.
- **Approve**: a second staff member with `access.emergency.approve`. The requester gets 403 `self_approval_denied`, audited as `access.denied`; a row that is not pending answers 409. The window restarts at approval and ends `min(durationMinutes, 240)` minutes later. One pino `warn` line `{ action: 'emergency_access.approved', emergencyAccessId }` feeds the `EmergencyAccessActivations` metric filter of the observability stack.
- **Deny** deletes a pending row (204, `emergency_access.denied`); **revoke** closes an active one (`emergency_access.revoked`, 409 `emergency_not_active` otherwise).
- **Read**: `GET /ops/emergency-access/:id/records/:recordType/:recordId` (`conversation` or `intake`) needs an operations session and a record that belongs to the caller, then `require(principal, 'records.read.emergency', resource)`, then the named record itself must be active and cover the resource (403 `emergency_not_active`). It returns the same view as the professional route and audits `emergency_access.used`. There is no list of records (CONF-R-09).

`findActiveEmergencyAccess` ignores unapproved rows and the engine checks `expiresAt` on every read, so an expired window is denied before the sweeper (section 14) closes the row. Request and approve are in the `emergency` step-up class (5 minutes), the read in `restricted` (15 minutes). Every write runs in `withTransaction` with its audit call and invalidates the requester's principal.

The emergency screen. Admin `/emergency` (`features/emergency/`; nav item and list need `audit.read`, which only `platform_owner` holds) lists the records with status (pending, active, expired, revoked; filterable), requester ("(you)" for the caller), approver, scope, purpose and the window ("N min from approval" while pending, a countdown while active). Request access takes the purpose (10 to 1000 characters), the scope kind and id and a duration of 15, 60, 120 or 240 minutes. Approve and Deny appear only on someone else's pending request and only with `access.emergency.approve`; on one's own request the row says it waits for another staff member, and the API refuses self-approval anyway (403 `self_approval_denied`, translated). Revoke appears on active rows. Open record appears only on the caller's own active access: the dialog states "This access is audited" before and after a read, takes the record type and id, reads through `withStepUp` and shows the conversation or the intake read-only; the content is validated against the professional schemas, kept out of the query cache and dropped when the dialog closes. Request, decision and read failures show translated copy (`self_approval_denied`, `emergency_not_active`, `conflict`, `not_found`, `forbidden`, `validation`, 429, `unknown`).



Admin `/emergency` for Nikola Đorđević: a pending request by another staff member (Approve, Deny) and his own active access (Open record, Revoke). Both rows were staged in the e2e database for the screenshot; their purposes say so.



The audited record view after reading `conv_sample` under the active access.

12. Audit log

`AuditWriter.record()` (`src/audit/audit-writer.ts`) inserts into `audit_events` with the category derived from the action (`audit-events.ts`: `auth.*` and `user.suspended` are SECURITY, `grant.*` and `access.denied` AUTHORIZATION, administration ADMIN, content CONTENT_ACCESS), the retention horizon from

`retention_policies` (10-minute cache, defaults per tier), a keyed hash of the IP (HMAC with `IP_HASH_SALT`, falling back to the session encryption key, so it cannot be reversed from a table of all addresses) and the parsed user-agent family. Details are sanitised: forbidden keys (tokens, passwords, message bodies, intake text, notes, voucher codes, raw email) are dropped and strings truncated. `record()` never throws in production; tests set `strictMode`. The table is range-partitioned by month and protected by triggers that reject UPDATE and DELETE; the application role also lacks those privileges. `pino-redaction.ts` redacts the same fields from HTTP logs. Add a new action to `AUDIT_ACTIONS` before using it; the type will not compile otherwise. `record()` writes through its own connection, not the caller's transaction: in strict mode a failed audit write rolls the caller's transaction back, otherwise the change commits and the writer logs the failure.

13. WebSockets

`NestSelfIoAdapter` (`src/ws/nestself-io.adapter.ts`, installed in `main.ts`) serves Socket.IO under `/api/ws`. The handshake middleware (`ws-auth.service.ts`) checks the `Origin` against the allowed surfaces, then authenticates either the web session cookie (same code path as HTTP) or a mobile ticket from `POST /auth/ws-ticket`: 30-second TTL, hashed, consumed atomically with `GETDEL` (`ws_ticket:{hash}`). The gateway (`events.gateway.ts`) exposes `subscribe { room, action, resource }` and `unsubscribe`; each subscription runs `authorize()` (`subscription-authorizer.ts`) and denied rooms emit `subscribe:denied`. `ws-invalidation.service.ts` listens on the `invalidate` channel and, for connected sockets of that user, reloads the principal, disconnects suspended or deactivated users and leaves rooms whose grants are gone (`subscription:revoked`). `ws-connection-registry.ts` tracks sockets per user.

14. Background jobs

Two kinds of job live in `src/jobs/`.

Jobs on behalf of a principal. `JobAuthorization` (`job-authorization.ts`) wraps BullMQ processors. A job payload carries `{ principalId, context, action, resource, data }` and never tokens or grants (AUTHZ-R-54). At execution the wrapper reloads the principal, rebuilds the context and calls `authorize()`; an unknown, suspended or denied principal completes the job as `skipped` (not failed, so no retry) with an `access.denied` audit event, while allowed jobs run the handler and audit with `actorKind: 'JOB'`. Feature modules that enqueue work (exports, imports) wrap their processors with it.

System maintenance jobs. `MaintenanceJobsService` (`maintenance-jobs.service.ts`) holds idempotent jobs that run in system context (no tenant setting) and audit with `actorKind: 'JOB'`, `actorUserId: null`:

Job	Schedule (local)	What it does
`audit.ensure-partitions`	01:00	Calls `audit_events_ensure_partitions(3)` so the next three months' partitions exist (D-21)
`grants.reconcile`	02:15	Closes grants whose `valid_until` passed (`grant.expired`), revokes delegations whose parent is revoked (`grant.delegation_revoked`), revokes organization-scoped grants whose subject has no ACTIVE membership (`grant.revoked`, reason `membership_ended`), then invalidates each affected principal and every delegatee below a closed grant (T-2.10, D-17)
`mobile.prune-stale-families`	02:30	Scans `user_mobile_families:*`; a family whose `mobile_family:{jti}` key has expired (30 days unseen) is denylisted in `revoked_family:{jti}` for 90 days, removed from the set and audited as `auth.mobile_family.revoked` (AUTHZ-R-64)
`sessions.reconcile-ttl`	02:45	Scans `session:*`; a record whose `absoluteExpiresAt` has passed (its key outlived its TTL) is destroyed through `SessionService.destroyOwnedByHash`, which revokes the refresh token, removes the hash from `user_sessions:{userId}` and publishes the invalidation. Malformed records are counted and left alone; a record whose destruction fails is counted in `failed` and retried on the next run (SESS-R-16)
`emergency.sweep-expired`	every 15 minutes	Sets `revoked_at` on approved emergency access whose `expires_at` has passed, audits `emergency_access.expired` and invalidates the requester. The engine already denies an expired window, so the sweep only closes the record

`JobSchedulerService` (`job-scheduler.service.ts`) registers these as BullMQ job schedulers on the `nestself-maintenance` queue (its own ioredis connection to `REDIS_URL`, cron in `JOBS_TIMEZONE`, default `Europe/Belgrade`, three attempts with exponential backoff) and runs a worker with concurrency 1. It is started only by the worker process: `src/worker/main.ts` boots the application context without an HTTP listener and calls `start()`; the ECS `worker` service runs `node dist/worker/main.js` from the same image as the API, locally `pnpm --filter @nestself/api worker:dev`. Without `REDIS_URL` the worker exits with an error. The worker boots the same `AppModule` as the API, so it also needs the API's identity provider settings and session encryption key: `sessions.reconcile-ttl` revokes refresh tokens through `SessionService`. `pnpm --filter @nestself/api job:run <name>` builds with `nest build` and executes one job immediately through `src/worker/run-job.ts` (`node dist/worker/run-job.js <name>`), printing its report, which is how to verify a job after a deploy or catch up after downtime. `KeyValueStore.scan()` exists for these jobs only; request paths never call it.

Local job tooling goes through the Nest compiler. Nest resolves constructor dependencies from decorator metadata, which `tsx` (esbuild) does not emit, so `worker:dev` is `nest start --entryFile worker/main --watch` and `job:run` is `nest build && node dist/worker/run-job.js`; the runner moved from `scripts/` to `src/worker/run-job.ts` so the build compiles it. Under `tsx` both failed with `Nest can't resolve dependencies of the AuthorizationService (?, ...)` until 2026-10-24. The other `tsx` scripts (`db:migrate`, `db:seed`, `db:backfill-grants`) do not boot Nest and are unaffected.

Unit tests (`maintenance-jobs.service.test.ts`) use mocked repositories and the in-memory store; `test/jobs.e2e.spec.ts` runs the reconciliation and partition jobs against the real database and checks the audit trail and the application role's permission to create partitions; `test/emergency.e2e.spec.ts` runs the emergency sweep.

15. Web clients

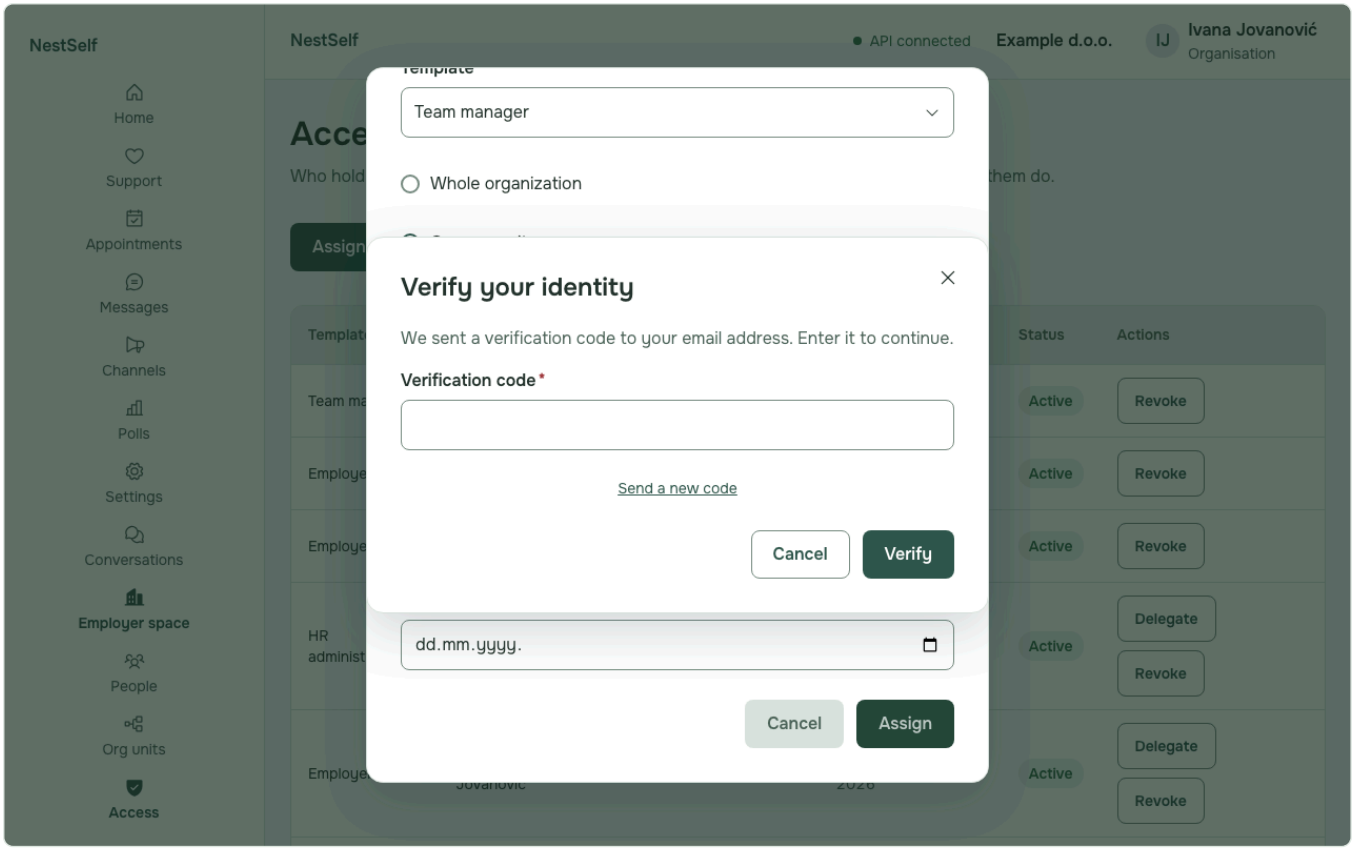
`packages/api-client/src/client.ts` works in cookie mode (default): `credentials: 'include'` and the CSRF cookie copied into `X-CSRF-Token` on mutating requests. `auth.ts` exposes typed functions for every endpoint (`discover`, `emailOtpStart`, `emailOtpVerify`, `password`, `challenge`, `totpEnrol`, `me`, `switchContext`, `signOut`, `listSessions`, `invitationAccept` ...) validated with the contracts schemas.

`apps/workspace/src/auth/` and `apps/admin/src/auth/`: `AuthProvider` calls `GET /auth/me` on mount and window focus and exposes `status`, `principal`, `effectiveActions` (computed by the API, section 9) and `stepUp()` through `useAuth()`; a 401 flips to unauthenticated. `sample-auth-provider.tsx` serves sample mode with `SAMPLE_PRINCIPALS`. `screens/sign-in-page.tsx` drives the identifier-first flow with one screen per step (`identifier`, `otp`, `password`, `new-`

`password`, `totp`, `totp-enrol`); `map-auth-error.ts` maps error codes to localized messages (`packages/i18n/locales/*/auth.json`), with `error.unknown` for codes it does not know. Routes under `routes/_authenticated/` redirect to `/sign-in` without a principal and gate sections with `can()` (for example `_hr.tsx` requires `program.overview`); `routes/_public/invite.$token.tsx` handles invitation links.

Route guards wait for the session. The guards (`_authenticated` in both apps; `_hr`, `_manager` and `_hr/hr/access` in the workspace) await `resolveRouteAuth(context.auth)` from `auth/route-auth.ts` before deciding. On a full page load the router context starts without a principal; in api mode it carries `ensure()`, which resolves once the first `/auth/me` has settled, and the provider reuses that same request. A deep link to a signed-in page therefore opens the page instead of bouncing to `/sign-in`; a 401 still redirects, with `search.return` set to the path.

Step-up in the browser. `stepUp()` opens `StepUpDialog` (`auth/step-up-dialog.tsx`), rendered once inside the provider; concurrent callers share one dialog and one promise. The dialog calls `stepUpStart()` when it opens, takes the code in `OtpInput` with the email-code length bounds, calls `stepUpVerify()`, and maps failures into `FormErrorSummary` and the field message: `invalid_code`, `invalid_challenge`, 429, `network`, `forbidden` from start (`stepUp.unavailable`) and `error.unknown` for anything else. `withStepUp(stepUp, fn)` (`auth/with-step-up.ts`) runs a mutation and, on 403 `step_up_required`, awaits `stepUp()` and retries once. When the user cancels it throws `StepUpCancelledError`; callers treat that as "declined" (the access dialogs close without an error) and `handleApiError` ignores it. The sample providers resolve `stepUp()` to `true`.



The step-up dialog over the assign dialog. For the screenshot the browser answered the first create with the API's own 403 `step_up_required` body, as the server does once the session is older than 30 minutes; the dialog then started a real challenge against the stub API.

Access screens. `createOrgAdminApi(client, { kind: 'org' | 'ops' })` in `@nestself/api-client` serves both apps; every mutation runs through `withStepUp`, and API codes map to translated copy under `access.errors.*` with `unknown` as the fallback.

- Workspace `/hr/access` (`features/access/`): the nav item and the route need `access.preview` and, because the route sits under `_hr`, `program.overview`. The screen switches to the organization context of the user's single active membership and shows the organization's grants. Assign (`access.grant`) offers organization templates at organization or org-unit scope with the required conditions; revoke (`access.revoke`) asks for a reason; delegate (`access.delegate`) hands a subset of one's own grant to another member; the effective-access panel previews any member. Transfer ownership appears only for the holder of the active owner grant and maps `owner_exists`, `subject_not_member` and `forbidden` to translated copy; after a transfer the owner-only controls disappear.
- Admin `/organizations/:orgId/access` (a tab next to Overview) and `/access` (platform grants): both need `access.preview`, switch to the operations context and use the ops API. The organization page assigns at organization scope only, because the ops API has no org-unit route; org-unit grants are HR's job in the workspace. It also transfers ownership through the staff route, which works when the organization has no owner yet. The platform page assigns `operations` templates at their own scope, revokes platform grants through `/ops/grants/:grantId/revoke` (the shared revoke dialog; a note says delegated grants go too) and shows user ids rather than names in api mode. Both web apps' query clients fail 4xx answers at once (`src/query-retry.ts`) and retry only 5xx, network and other errors, up to three times. Both pages offer the user-access lookup (`/ops/users/:userId/access`).
- In sample mode the screens read `SAMPLE_GRANTS` and disable every mutation with a note.
- The grant table recomputes each row's status on every render, so a grant created a moment ago shows Active, not Scheduled.

The dialog, the helpers, `route-auth.ts` and the access screens exist once per app, because a shared package would need an install.

Popups inside dialogs. In `@nestself/ui`, Select and Menu popups opened inside a Dialog, AlertDialog or Sheet layer above it: the dialog's portal element sets `--z-popup` to one above `--z-dialog` (`src/lib/layers.ts`). Before 2026-10-24 the popup sat below the dialog and a pointer could not choose an option.

16. Mobile client

`apps/mobile/src/auth/`: `cognito-public-client.ts` talks to Cognito directly (`InitiateAuth` with `USER_AUTH` for email codes, refresh, revoke), `sso.ts` prepares the PKCE authorization-code flow through `expo-auth-session`, `token-store.ts` keeps tokens in `expo-secure-store` (device-only, unlocked access), `jwt.ts` decodes claims, `auth-state.ts` is the reducer and `auth-provider.tsx` orchestrates sign-in, refresh, sign-out and the sample mode. API calls use the bearer mode of the shared client; WebSocket connections fetch a ticket first. Mobile settings come from `EXPO_PUBLIC_COGNITO_*`. The email-code start sends the device locale as `ClientMetadata.locale`. There is no step-up screen yet: a `step_up_required` answer has no UI on mobile, although the API side accepts bearer callers (section 9).

17. Infrastructure and the Cognito rollout

`infra/cdk/lib/auth-stack.ts` defines the user pool `nestself-<stage>` (Essentials plan, email OTP and password first factors, optional TOTP, no self sign-up), three app clients (confidential web with secret, public mobile with PKCE, reserved machine-to-machine), the `CustomMessage` and `PreSignUp` Lambda triggers (`lib/triggers/`), the web client secret and the session encryption key in Secrets Manager, and optional WAF, SES and custom domain for staging and production. Outputs: `UserPoolId`, `WebClientId`, `MobileClientId`, `CognitoDomain`, `CognitoIssuer`, secret ARNs.

The CDK app contains only this stack unless `-c platform=true` is passed, which keeps the rollout to Cognito-only until the web applications ship (`infra/cdk/README.md`, "Rollout order"). To connect the API to a real pool:

```
pnpm --filter @nestself/cdk exec cdk bootstrap aws://<ACCOUNT_ID>/eu-central-1
pnpm --filter @nestself/cdk exec cdk deploy NestSelf-dev-Auth -c stage=dev -c account=<ACCOUNT_ID>
node infra/cdk/scripts/env-from-outputs.mjs dev >> apps/api/.env     # COGNITO_* and the secret
```

The step-by-step account setup, credentials, IAM permissions and every environment variable are in `CDK-setup.md` at the repository root. Then set `AUTH_MODE=cognito`, `REDIS_URL` and `SESSION_ENCRYPTION_KEY` (from the stack secret) in `apps/api/.env`. Users must exist in the pool: create them through an invitation (the accept flow calls `AdminCreateUser`) or with the AWS console for a first test; the API links them by `sub` on first sign-in only when an `external_identities` row exists, so onboarding goes through invitations. `env-from-outputs.mjs <stage> --mobile` prints the `EXPO_PUBLIC_*` values for the app (`EXPO_PUBLIC_COGNITO_CLIENT_ID` is the name `apps/mobile/src/auth/config.ts` reads).

The API process needs AWS credentials for the Cognito admin calls (`AdminInitiateAuth`, `AdminRespondToAuthChallenge`, `AdminCreateUser`, `AdminDeleteUser`, `AdminDisableUser`, `AdminEnableUser`, `AdminGetUser`, `AdminSetUserPassword`, `AdminUserGlobalSignOut`, `GetTokensFromRefreshToken`, `RevokeToken`, `ListUsers`, `DescribeUserPoolClient`) and, when `SES_FROM_EMAIL` is set, `ses:SendEmail`. In AWS these come from the ECS task role (`infra/cdk/lib/constructs/ecs-task-roles.ts`); locally from the default credential chain (`AWS_PROFILE`). The compute stack's `worker` service runs `node dist/worker/main.js` (section 14) with the worker task role.

17.1 The dev pool as deployed

First deployed 2026-10-24 into a dedicated account. Current coordinates:

Account / region	`634972095709` / `eu-central-1`
Stacks	`CDKToolkit`, `NestSelf-dev-Auth`
User pool	`eu-central-1_g6OGYONQ0`, hosted domain `nestself-dev`
App clients	`nestself-web-dev`, `nestself-mobile-dev`, `nestself-m2m-dev`
Secrets	`nestself-dev/cognito/web-client-secret`, `nestself-dev/api/session-encryption-key`
Triggers	`nestself-dev-cognito-custom-message`, `nestself-dev-cognito-pre-sign-up`

Three things about this stack are easy to get wrong.

The username is the email address. `auth-stack.ts` sets `signInAliases: { email: true }` without `username: true`, which CDK compiles to `UsernameAttributes: ['email']`, not `AliasAttributes: ['email']`. `AdminCreateUser` therefore **requires** an email as `--username` and rejects a plain handle with `Username should be an email`. Cognito then generates the internal username itself, and it is the same UUID as `sub`, so a manually linked `external_identities` row gets that one value in both `subject` and `cognito_username`:

```
aws cognito-idp admin-create-user --user-pool-id eu-central-1_g6OGYONQ0   --username <address> --user-attributes Name=email,Value=<address>
Name=email_verified,Value=true   --message-action SUPPRESS
```

`--message-action SUPPRESS` keeps AWS from sending anything at creation time; the first mail is the sign-in code. The `PreSignUp` trigger refuses self sign-up but does not block `AdminCreateUser`.

`USER_ID_FOR_SRP` is absent in the EMAIL_OTP challenge. `AdminInitiateAuth` with `PREFERRED_CHALLENGE: EMAIL_OTP` returns only `CODE_DELIVERY_DELIVERY_MEDIUM` and `CODE_DELIVERY_DESTINATION` in `ChallengeParameters`. `startEmailOtp` reads `ChallengeParameters['USER_ID_FOR_SRP'] ?? input.email`, so the fallback always wins and the email is the username in both the initiate and the respond call. That keeps `SECRET_HASH` consistent across the two calls, which is what the confidential web client requires.

The email code is eight digits, not six. See section 6. Anything that validates or caps the length must use `EMAIL_OTP_MIN_LENGTH` / `EMAIL_OTP_MAX_LENGTH`; a six-digit `maxLength` silently truncates and produces `CodeMismatchException`.

`AWS_PROFILE` in `apps/api/.env` needs the ConfigModule merge. `envSchema` is a plain `z.object`, which strips undeclared keys, and ConfigModule's `validate:` branch assigns only what the function returns back onto `process.env` (its `validationSchema:` branch merges them back, `validate:` does not). Before 2026-10-24 that silently dropped `AWS_PROFILE`, the SDK fell back to the machine's default profile, and the Cognito calls failed with a cross-account `AccessDenied` that still named the correct pool id. `validateEnv` now returns `{ ...config, ...appEnv }`, which also preserves `AWS_ACCESS_KEY_ID`, `AWS_SESSION_TOKEN`, `HTTPS_PROXY` and `NODE_EXTRA_CA_CERTS`.

Tearing the pool down takes three calls in order, because the stack enables deletion protection and Cognito refuses to drop a pool that still has a domain:

```
aws cognito-idp update-user-pool --user-pool-id <id> --deletion-protection INACTIVE
aws cognito-idp delete-user-pool-domain --user-pool-id <id> --domain <prefix>
aws cognito-idp delete-user-pool --user-pool-id <id>
```

18. Configuration reference

Validated in `apps/api/src/config/env.ts`; documented in `docs/environment.md`.

Variable	Purpose
`AUTH_MODE`	`stub` (default; refused in production) or `cognito`
`DATABASE_URL`, `DATABASE_POOL_MAX`	PostgreSQL connection for the API (a `nestself_app` member in Docker and AWS)
`REDIS_URL`	Required in cognito mode; optional in stub mode (in-memory store)
`JWT_SECRET`, `JWT_EXPIRES_IN`	Stub-mode bearer tokens only; removed after cutover
`COGNITO_REGION`, `COGNITO_USER_POOL_ID`, `COGNITO_WEB_CLIENT_ID`, `COGNITO_WEB_CLIENT_SECRET`, `COGNITO_MOBILE_CLIENT_ID`, `COGNITO_DOMAIN`, `COGNITO_ISSUER`	Pool coordinates; `env-from-outputs.mjs` prints them
`SESSION_ENCRYPTION_KEY`, `SESSION_ENCRYPTION_KEY_PREVIOUS`	32-byte hex keys for token encryption at rest; derived from `JWT_SECRET` in stub mode
`IP_HASH_SALT`	Key for the audit and session IP hashes; defaults to the session encryption key
`SESSION_COOKIE_NAME`, `CSRF_COOKIE_NAME`, `SESSION_COOKIE_INSECURE_DEV`	Cookie names and the localhost switch
`SESSION_LIFETIME_OVERRIDES`	JSON overrides per `population:context`
`MARKETING_URL`, `WORKSPACE_URL`, `ADMIN_URL`, `API_PUBLIC_URL`	Allowed origins for CORS, CSRF and WebSocket handshakes; invitation links
`TRUST_PROXY_HOPS`	Trusted proxy depth for client IP extraction
`SES_FROM_EMAIL`	Enables the SES invitation mailer
`JOBS_TIMEZONE`	Worker process: IANA timezone of the nightly job schedules (default `Europe/Belgrade`)
`FAKE_IDP_FIXED_OTP`	Stub mode: an always-accepted six-digit code
`TEST_DATABASE_URL`	Integration tests; suites create `<name>_schema`, `<name>_auth`, ... databases
`AWS_PROFILE`, `AWS_ACCESS_KEY_ID`, `AWS_SESSION_TOKEN`, `HTTPS_PROXY`, `NODE_EXTRA_CA_CERTS`	Not declared in `envSchema`, but passed through to `process.env` for the AWS SDK and the HTTP stack; see the merge in `validateEnv` and section 17.1

Anything not declared in `envSchema` is still forwarded to `process.env`, because `validateEnv` returns `{ ...config, ...appEnv }`. Removing that spread re-breaks every library that reads `process.env` directly instead of through `ConfigService`.

Redis keys and TTLs are listed in `apps/api/src/redis/redis-keys.ts`.

19. Testing

- Unit tests sit next to the code (`*.test.ts`, `*.spec.ts`) and mock repositories and services. The authorization engine has unit, property and golden-table tests; `step-up-freshness.test.ts`, `grant-rules.test.ts` and `effective-access.test.ts` cover the phase 2 rules.
- Integration suites in `apps/api/test/` boot the real application against a disposable PostgreSQL (`TEST_DATABASE_URL`, one database per suite via `test/db/test-database.ts`), use the fake identity provider and the in-memory store (the test environment ignores `.env` and `REDIS_URL`). Without `TEST_DATABASE_URL` they skip; locally use `TEST_DATABASE_URL=postgresql://nestself:nestself_dev@localhost:54329/nestself_test`.
- **Per-worktree test database names.** Each suite's database is `<base>_<prefix>_<suffix>`: `<base>` is the database in `TEST_DATABASE_URL`, `<suffix>` names the suite, and `<prefix>` is `TEST_DATABASE_PREFIX` or, when that is unset, the first eight hex characters of the SHA-1 of the repository root path (`apps/api/test/db/test-database-name.ts`). Git worktrees of the repository, where several agents often work at once, therefore get separate databases on one server and never drop each other's. Names over PostgreSQL's 63-byte limit shorten the base; old databases are not removed automatically.

Suite	Covers
`auth.e2e.spec.ts`	Sign-in, CSRF, context switch, profile, sessions, lockout, the three-attempt email code, step-up start and verify, `effectiveActions` in `/auth/me`
`ops-clients.e2e.spec.ts`	`/ops/orgs/:orgId/members` and `/invitations` through the CLIENT assignment
`grants.e2e.spec.ts`	Organization grant administration, delegation bounds and depth, delegatee access and its revocation, approval authorities, the write throttle
`ops-grants.e2e.spec.ts`	Platform grants, ops organization grants, user access
`professional.e2e.spec.ts`	Confidential reads by participant and assigned expert, denials and their audit, step-up on a stale session
`emergency.e2e.spec.ts`	Request, self-approval refusal, approval, record read, expiry before the sweep, sweep, deny, revoke

Suite	Covers
`members.e2e.spec.ts`, `invitations.e2e.spec.ts`, `org-admin.e2e.spec.ts`, `ws.e2e.spec.ts`	Their areas; members also proves delegatee invalidation on membership end
`health.e2e.spec.ts`, `jobs.e2e.spec.ts`	Health with database and Redis; the maintenance jobs
`db/schema.spec.ts`	Six migrations, partitions, append-only triggers, RLS on all sixteen tenant tables, the seed's stale-grant cleanup
`db/authorization-repository.spec.ts`	Grant repository methods, delegation walks and emergency access queries under RLS
`db/backfill-grants.spec.ts`	The legacy role backfill and the `0005` guard

- Browser journeys live in `e2e/auth/` and run with `pnpm e2e:auth` (`playwright.auth.config.ts`, project `auth-desktop`): ten tests. `workspace-sign-in.spec.ts` (email-code sign-in, sign-out through the account menu, guard redirect, wrong code), `admin-sign-in.spec.ts` (password and authenticator), `hr-access.spec.ts` (Ivana's grant table, preview, delegation, assign and revoke), `account.spec.ts` (Ivana's display name by deep link, change and header, sessions list with this browser marked, sign out everywhere ending a second browser too), `hr-people.spec.ts` (invite and revoke a fictional address, move Ana Nikolić to Sales and back, add and rename an org unit, open and cancel End membership), `admin-ops.spec.ts` (organizations list, the sample organization's People tab, an emergency request by Nikola with Approve and Deny hidden on his own row, approval by Milica in a second browser, the audited record view reading `conv_sample`, the user lookup offering Suspend for `user_jovan`, cancelled) and `expert-cases.spec.ts` (Milica opens `case_sample` from `/cases` and reads its messages without a step-up). `admin-ops.spec.ts` stages `platform_owner` grants for Nikola and Milica (`grant_user_nikola_platform_owner_e2e`, `grant_user_milica_platform_owner_e2e`) because the seed has no platform owner and emergency approval needs a second one; the fake identity provider lives in the API process, so a new staff account cannot be created from the browser test. `e2e/auth/support.ts` holds the sign-in helpers, the per-user authenticator secrets in `e2e/.auth-state/` and the staging helper, which also drops the user's cached principal from Redis (Redis outlives the API process). They need the API in stub mode with `FAKE_IDP_FIXED_OTP=123456` and the web apps in `api` data mode; the config starts all three. See `e2e/README.md`, which also explains why a cognito-mode API on port 4000 must be stopped first.
- **The e2e database.** The stub API never uses the developer database in `apps/api/.env`. It runs against `nestself_e2e_<prefix>` on the `TEST_DATABASE_URL` server (same prefix rule as above), or exactly `E2E_DATABASE_URL`, which must name a `nestself_e2e_*` or `nestself_test_*` database. `e2e/global-setup.ts` drops it (`WITH (FORCE)`), recreates it, migrates it and runs the sample seed on every run through the API's own scripts (`scripts/reset-database.ts`, `migrate.ts`, `seed.ts`) with `DATABASE_URL` overridden; the API web server command first runs the same file with `--ensure`, because Playwright starts web servers before global setup and the API pings its database on boot. `reset-database.ts` refuses any other database name. Without `TEST_DATABASE_URL` or `E2E_DATABASE_URL` the run stops with a message before a server starts.
- **Guide screenshots.** `pnpm docs:auth:screenshots` runs the `screenshots` project of the same config (`e2e/screenshots/*.shot.ts`, 1280×800, light, device scale 1, reduced motion) against the same stack and writes the PNGs in `docs/auth/screenshots/`; `pnpm docs:auth:pdf` rebuilds both PDFs (`docs/auth/pdf/build.mjs` resolves relative images against the Markdown file and fails on a missing one). Four files: `admin.shot.ts`, `operations.shot.ts`, `workspace-screens.shot.ts` and `workspace.shot.ts` (they run in name order, so `admin.shot.ts` enrols Nikola first). Nothing is created through the screens except the import in `workspace-screens.shot.ts`, which creates two invitations for fictional `new.colleague.*@example.com` addresses in the disposable database so the result and the pending list have rows; every other dialog is cancelled or only read. Before any capture every shot file removes the real developer account `user_dragan` from the disposable database (section 21), and `shot()` refuses a capture that shows it. What is staged, and the captions say so: a `platform_owner` grant for Nikola (`admin.shot.ts` and `operations.shot.ts`), because the sample data has no platform owner and the admin pages need it; two emergency access rows in `operations.shot.ts` whose purposes start with "Staged for the guide screenshots" (one pending request by Milica, one approved access of Nikola's for `case_sample`), because an approval needs a second platform owner; and in `workspace.shot.ts` one create answered in the browser with the API's `step_up_required` body to open the step-up dialog. The next reset removes everything. The enrolment screenshot needs a fresh API process, so stop any API on port 4000 first.
- **Visual baselines.** The non-gated suite (`pnpm e2e`, `playwright.config.ts`, `e2e/__screenshots__/`) runs the apps in sample mode. Every authenticated page has the account menu and the context picker in its header since the UI-completeness plan. On 2026-10-25 the workspace and admin entries were checked: below 480px the header now hides the API status, so it fits 320px (it overflowed by 67px) and the mobile baselines did not move; only the four settings baselines (new account, sessions and authenticator sections) were updated and the missing sign-in baselines added. The same run also fixed sample-mode deep links with `?role=` (`/hr?role=hr_admin` was redirected home, because the guards of the first navigation saw no principal; `sampleRouteSnapshot()` in `auth/sample-auth-provider.tsx`). A workspace with `VITE_DATA_SOURCE=api` in `.env.local` must run with `VITE_DATA_SOURCE=sample` set in the shell for this suite (Vite lets the shell override `.env.local`), and a reused workspace server in api mode must be stopped first.
- `pnpm --filter @nestself/api test` runs everything; `pnpm test` runs the whole repository. CI (`.github/workflows/ci.yml`) provides Postgres and Redis service containers.

20. Recipes

Add a protected route. Put the handler in the right controller, pick the action from `ACTIONS`, and decorate with `@Authorize(ACTIONS['x.y'], orgResourceFromContext('thing'))` (or `operationsResource`, or `operationsOrgResource` under `/ops/orgs/:orgId`). Step-up follows from the action's class; nothing to add. Access-administration writes also carry `@Throttle(WRITE_THROTTLE)`. When the decision needs row data (owner, case, confidentiality), mark the route `@Authenticated()`, load the row in the service and call `AuthorizationService.require()` before returning anything. Read the body with `@Body(new ZodValidationPipe(Schema))`; schemas live in `packages/contracts/src/org-admin.ts`. Do writes inside `dbService.withOrgScope(orgId, tx => repo.method(..., tx))`, audit through `AuditWriter`, and invalidate caches that your change affects (`PrincipalLoader.invalidateUser`, or `org_tree:{orgId}` for tree changes). The default-deny test tells you if you forgot the decorator; add an e2e case to the matching suite.

Add an action or template. Extend `ACTIONS` in `packages/contracts/src/actions.ts` (and `CONFIDENTIAL_ACTIONS` or `STEP_UP_CLASS_BY_ACTION` where relevant), add it to the template in `templates.ts`, add golden-table tuples in `apps/api/src/authz/golden-table.json`, and re-run the seed so sample grants pick it up.

Add a table or column. Edit `apps/api/src/db/schema/*.ts`, run `db:generate`, review the SQL in `apps/api/drizzle/`, add a `pgPolicy` with `tenantPredicate()` if the table has an organization column, run `db:migrate`, and extend the matching repository. Partitions, triggers and roles go in a `--custom` migration.

Add an audit action. Append to `AUDIT_ACTIONS` in `apps/api/src/audit/audit-events.ts` and map its category in `resolveCategory`; the spec file enumerates them.

Revoke someone's access programmatically. Write the audit event first, then `AccessRevocationService.revokeAllAccess(userId, { sessionsForOrgId?, orgIds? })`.

Call a sensitive action from a web screen. Wrap the mutation in `withStepUp(useAuth().stepUp, () => api.call(...))`, map its `ApiError` codes to translated copy, and treat `StepUpCancelledError` as the user declining (close or leave the form without an error).

Debug a denial. Call `explain(principal, action, resource, ctx)` from `src/authz/explain.ts` in a test, or read the `access.denied` audit row: `details.reason` carries the engine's denial reason.

21. Known gaps and next steps

Phase 2 (`docs/superpowers/plans/2026-10-24-auth-phase-2.md`) is done; `docs/tasks/auth/phase-2-readiness.md` has the status of every T-2.x task. What remains open, grouped by kind.

Owner decisions

Resolved on 2026-10-24: organization grants are administered by the single `employer_owner`, who can transfer the role, and by NestSelf staff (section 11.1, "Organization ownership"; strategy 4.4 amendment).

- **Authenticator step-up for the `emergency` class.** Every class steps up with an email code (section 9); the strategy asks for an authenticator for `emergency`. Doing that means a password-plus-authenticator step-up or passkeys.
- The `presenter` template (GAP-H02), retention periods (L-02, L-03), who may approve emergency access and whether a second approver is required (default: `platform_owner`, one approver), and the staging date. These carry over from `phase-2-readiness.md` section 4.

Blocker before any conversation write path

- `conversations.case_id` and `intake_records.case_id` are free text with no organization binding, and the assignment match compares `targetId` only. Before any route creates conversations or intakes, add a `cases(id, org_id)` table with foreign keys from both tables, or bind assignments to an organization. `schema/professional.ts` carries a `TODO(phase-2-readiness)` at both columns.

Not built in phase 2

- **Mobile step-up UI.** The API accepts bearer callers; the app has no screen for `step_up_required`.
- **Screens still missing.** Approval authorities (`/orgs/:orgId/approval-authorities`) and changing a grant's conditions or validity (`PATCH /orgs/:orgId/grants/:id`) have API routes but no screen. Mobile keeps sign-in, settings and sign-out only; the administration screens are web only by decision (section 3.1).
- **T-2.8** security baseline, SCPs and the production account (waits for staging); **T-2.9 and T-2.11** file delivery and export authorization (nothing to authorize yet); **T-2.12** the weekly Cognito user export.
- **Remaining jobs:** the retention sweeper (needs L-02 and L-03), the audit export to S3 with Object Lock (AUDIT-R-06), the SSO certificate-expiry check (SSO-R-14). Each is a method on `MaintenanceJobsService` plus a schedule entry.
- **Enterprise SSO** (phase 3): connection management, home-realm discovery and the callback are stubs. Federated users get 403 `forbidden` on step-up start until then.
- **Community feature retirement:** `posts`, `polls`, `poll_options`, `poll_votes`, `legacy_teams` and `memberships.team_id` are tenant-isolated but deprecated; the contracts' `OrgRole` and `PlatformRole` remain for sample mode.
- **Compliance track** before the first real employee signs in: privacy notices, processing agreement, breach procedure, data-subject-rights endpoints (T-C.4).

To verify against the live pool

- Whether Cognito answers the third `CodeMismatchException` within one `Session` with `NotAuthorizedException` instead. The fake provider keeps the session alive for all three tries. If Cognito ends it sooner, the API treats that as any other provider error and ends the challenge, which is safe but gives the user fewer retries than the screens suggest.

Known limitations from the phase 2 reviews

- **Throttling.** Access-administration writes are limited to 10 per minute per route and client IP, not 10 per session across routes as AUTHZ-R-14 asks. Wrong step-up codes are bounded by the 5-per-minute route throttle and three tries per challenge only: there is no per-session `throttle:step_up` tracker and no `SignInThrottle` lockout for step-up. A provider-originated 429 shows different copy from the lock message.
- **Step-up edges.** Start answers a generic 403 `forbidden` for accounts without a native identity (a dedicated `step_up_unavailable` code would read better); a bearer principal without a token family is asked to step up but cannot start; an attempt to use another user's challenge is refused and audited (`reason: 'foreign_challenge'`). The dialog has no resend cooldown or in-flight guard and keeps Cancel enabled during verify.
- **Audit outside the transaction.** `AuditWriter` writes through its own connection, so outside strict mode a grant or emergency change can commit while its audit row failed (the writer logs it). A writer that joins the caller's transaction is the follow-up. The emergency sweeper has the same shape and stops its loop on the first error.
- **Grants and delegation.** Delegatee invalidation runs after the commit, so a failure leaves caches stale for up to the grants TTL (about a minute). A PATCH that shortens a parent's validity does not re-check its children (the engine denies them once the parent expires), and a PATCH that changes both conditions and validity checks children against the stored validity. Repeated identical delegations are accepted and self-delegation is not refused. `grants.service.ts` is about 580 lines and should split platform grants and subject access into their own services. `POST /ops/grants` does not check the subject's population. An approval-authority PATCH cannot clear `validUntil` or `maxAmountMinor`.
- **Invitations.** The two route families return slightly different 400 bodies.
- **Ownership transfer.** A new owner who already held a grant delegated from the old owner grant gets it re-parented under their own owner grant, which is redundant but harmless (they can revoke it). The API e2e cases for a 403 transfer probably stop at the guard; the service's own owner check is proven by unit tests only. Golden rows for the owner copy the template's actions by hand.
- **Confidential and emergency reads.** A missing id answers 404 before authorization and an existing denied one 403, so members and staff can tell whether an id exists. Pending emergency requests never lapse, the record list is ordered by `grantedAt`, which approval overwrites, and `listMessages` is unbounded.

- **Backfill.** Grant ids for a user's first organization still lack the organization; the HR admin chosen as `grantedByUserId` depends on a creation-time tie-break; an expert without approved credentials gets a grant without category conditions; the `0005` guard ignores `revoked_at` and validity. A legacy `MANAGER` without an org unit gets an organization-wide `team_manager` grant (reported in `widened` for review), which is broader than any unit they could have managed.
- **Web.** `/hr/access` sits under `_hr`, so it also needs `program.overview`: an employer owner without `hr_admin` cannot reach it (the sample owner holds both). Moving the route out of `_hr` is a small change. Member pickers load the first 100 active members; spending ceilings assume two-decimal currencies; the user-access lookup cannot be re-run with the same id. A stalled `/auth/me` keeps a deep-linked page waiting with no timeout, and a 500 or network failure on `/auth/me` looks like being signed out (only a console warning). The access dialogs have smaller form-polish items (revoke errors pinned to the reason field, focus on a disabled confirm).
- **UI-completeness follow-ups (2026-10-25).**
  - *Conversations do not name their intake.* Neither `ConversationSummary` / `ConversationResponse` nor the case carries an `intakeId`, so the expert types the intake record id. Add `intakeId` to the conversation responses, or a lookup by case.
  - *Ids instead of names.* Conversation participants, emergency requesters and approvers and the platform grant table show user ids, because those responses carry no display names and there is no staff user-name lookup. Add names to the responses.
  - *Import errors as English sentences.* `POST .../invitations/import` returns some row reasons as sentences (`Invalid template key: ...`, `Duplicate within batch`, `Org unit ... does not belong ...`); both apps match them by prefix. Return codes instead. The same response counts a refused duplicate or member row in `skipped` and in `errors`, so the summary reads "Skipped: 1. Not imported: 1." for one row.
  - *`cannot_suspend_self`: done.* The admin hides Suspend for one's own account and the API refuses it with 409 (`users.service.ts`). Ending one's own membership is still refused only by the UI; the API follow-up is a 409 `self_end`.
  - *`audit.read` for the emergency page.* The list route needs `audit.read`, which only `platform_owner` holds, so staff with `access.emergency.request` or `access.emergency.approve` but without `audit.read` (`security_admin`) never see the page. Whether they should, and with which list action, is a product decision.
  - *Picker caps.* Member pickers (assign, delegate, transfer, the admin organization overview's owner name) read the first 100 active members; the HR line-manager directory reads every page. A searchable picker is the follow-up.
  - *Assigned clients are not discoverable.* `/auth/me` exposes no CLIENT assignments, so a `client_operations` user reaches an assigned client only by link.
  - *Smaller items.* `organizations.status` is free text in the database but a four-value enum in the contract (add a CHECK); content already read in the emergency record view stays on screen if the window ends while the dialog is open; picker failures are toasts only; the sample-mode context switch does not change the navigation; the Playwright stub API reads `REDIS_URL` from `apps/api/.env`, so it shares Redis with a developer's API (the staging helper evicts the principals it changes).
  - *A real person in the sample data.* `SAMPLE_PEOPLE` carries the developer account `user_dragan` (Dragan Podrašćanin), added by the owner on 2026-10-24 to link the real dev pool and flagged `sample: true` like the fictional people. Guide screenshots show fictional people only: every shot file calls `removeDeveloperAccount()` (`e2e/auth/support.ts`), which deletes that account from the disposable e2e database before any capture, and `shot()` fails if the account's name or address is on screen. Member lists are ordered by random membership ids, so filtering screen by screen would not keep the row out reliably. The seed and the dev pool link are unchanged.
  - *Visual snapshot baselines.* The workspace and admin entries of `pnpm e2e` pass again since 2026-10-25 (section 19). The rest of that suite fails for reasons outside this plan: the marketing pages fail axe's `document-title` rule and their screenshots and 320px reflow differ from the baselines, and the suite's `testDir: './e2e'` also collects `e2e/auth/*.spec.ts`, which need the stub stack (`testIgnore` for `auth/**` would fix that). They need their own look.
- **Health.** The Redis indicator detects memory mode with `instanceof`, and its 3-second timeout path is untested.
- **Tests.** Several API e2e suites share one session across cases or depend on case order, and the `ageSession` and `extractSessionId` helpers are duplicated across specs.
- **Email-code locale.** Sign-in and resend send the screen's language (section 6), but step-up start and the invitation-acceptance code still call `startEmailOtp` without one, and a `custom:locale` attribute on the pool user always wins over `ClientMetadata` in the trigger.
- **Mobile authentication strength** is reported as `email_otp` without a second factor because that is the only first factor the mobile client allows; it must be revisited when mobile gains passkeys or SSO.
- **The email code length is AWS's, not ours.** Cognito's managed `EMAIL_OTP` challenge always sends eight digits: there is no pool setting, no CDK property and no CLI parameter for it (verified 2026-10-24 against `describe-user-pool`, `update-user-pool` and `UserPoolProps`). Shortening it to four or six means leaving the managed flow for `CUSTOM_AUTH` with Define, Create and Verify Auth Challenge triggers, where the code, its delivery through SES, its expiry and its attempt counter all become ours. That is a deliberate trade: shorter codes are friendlier to type and materially weaker (four digits is 10,000 combinations), and the throttle would have to tighten to compensate. Not started.